TRABAJO PRÁCTICO: DESARROLLO DE ALGORITMOS

Diagrama de flujo • Pseudocódigo • Prueba de escritorio • Codificación en C • Conclusiones

1. Datos informativos

Asignatura:	Teoría de la programación
Docente:	José Guamán
Estudiante(s):	Fabián Alejandro Gordillo Saraguro
Curso / Paralelo:	Ciclo 1, A
Fecha de entrega:	1/11/2026
Tema:	Desarrollo de algoritmos: representación gráfica, pseudocódigo, prueba de escritorio y codificación en C

2. Objetivo

Desarrollar los algoritmos propuestos por el docente aplicando las distintas formas de representación y construcción de soluciones: diagrama de flujo, pseudocódigo, prueba de escritorio y su implementación en el lenguaje de programación C.

3. Estructura del trabajo

Ejercicios avanzados

3.1.1. Enunciado del algoritmo

EJ-096

En una joyeria, construya un algoritmo que calcule la dosis de un medicamento segun el peso (real, kg) y la edad (entero) del paciente. Use constantes simbolicas DOSIS_ADULTO = 500.0, DOSIS_NIÑO = 250.0, EDAD_ADULTO = 18, PESO_MIN = 5.0. Evalúe las expresiones logicas (edad >= EDAD_ADULTO) y (peso >= PESO_MIN). Calcule la dosis con la expresion aritmetica (edad >= EDAD_ADULTO ? DOSIS_ADULTO * peso / 70 : DOSIS_NIÑO * peso / 30). Asigne el resultado a dosis. Muestre por pantalla la dosis recomendada. Ademas, calcule el valor minimo de todas las variables utilizadas. Identifique explicitamente los datos de entrada y la informacion de salida.

3.1.2. Análisis del problema

Datos de entrada

•peso
•edad

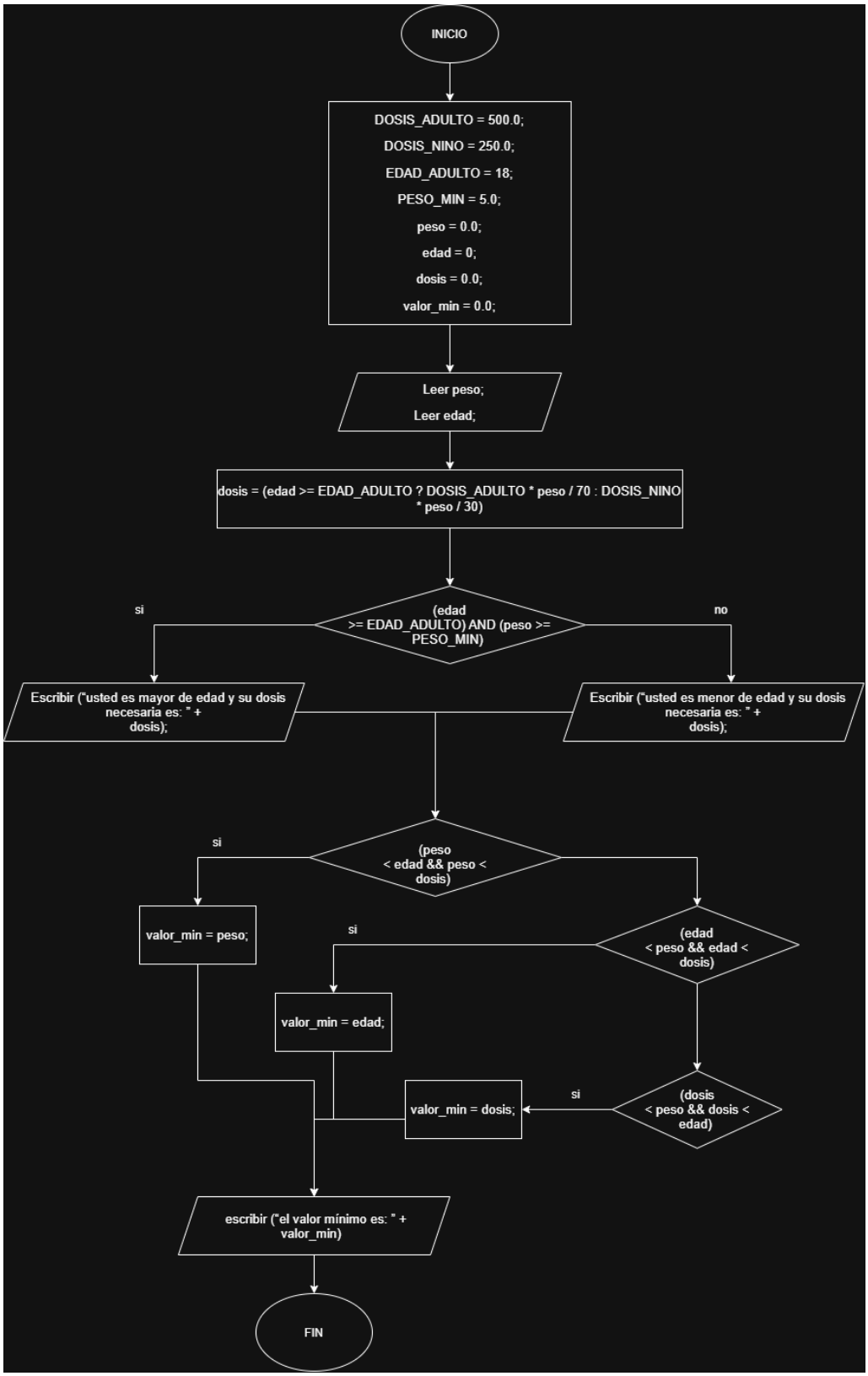
Proceso

1. declarar DOSIS_ADULTO = 500.0, DOSIS_NIÑO = 250.0, EDAD_ADULTO = 18, PESO_MIN = 5.0, peso = 0.0, edad = 0 y dosis = 0.0
2. leer peso y edad desde el teclado
3. evaluar (edad >= EDAD_ADULTO) AND (peso >= PESO_MIN)
4. calcular dosis con la expresión aritmética (edad >= EDAD_ADULTO ? DOSIS_ADULTO * peso / 70 : DOSIS_NIÑO * peso / 30), y asignarle el valor de dosis
5. calcular el valor mínimo de todas las variables usadas

Salida

mostrar si es mayor de edad y la dosis recomendada

3.1.3. Diagrama de flujo



3.1.4. Pseudocódigo

```
INICIO
//declaración de datos
DOSIS_ADULTO = 500.0;
DOSIS_NINO = 250.0;
EDAD_ADULTO = 18;
PESO_MIN = 5.0;
peso = 0.0;
edad = 0;
dosis = 0.0;
valor_min = 0.0;
//entrada
Escribir (“ingrese su peso en kg”)
Leer peso;
Escribir (“ingrese su edad”)
Leer edad;
//proceso
      dosis = (edad >= EDAD_ADULTO ? DOSIS_ADULTO * peso / 70 : DOSIS_NINO *
      peso / 30)
//evaluación y salida
Si (peso < edad && peso < dosis) entonces
      valor_min = peso;
si (edad < peso && edad < dosis) entonces
      valor_min = edad;
si (dosis < peso && dosis < edad) entonces
      valor_min = dosis;
Si (edad >= EDAD_ADULTO) AND (peso >= PESO_MIN) entonces
      Escribir (“usted es mayor de edad y su dosis necesaria es: ” + dosis);
Sino
      Escribir (“usted es menor de edad y su dosis necesaria es: ” + dosis);
      Escribir (“el valor mínimo de las variables es:” + valor_min)
FIN
```

3.1.5. Prueba de escritorio

Pasos	1	2	3	4	5	6
Casos	peso	edad	dosis	Valor mínimo	¿Mayor de edad y peso minimo?	Salida
Caso 1	70.2	18	501.43	18	Verdadero	Valor mínimo = 18 Es mayor de edad y su dosis es de: 501.43
Caso 2	60.0	12	500.00	12	Falso	Valor mínimo = 12 Es menor de edad y su dosis es de: 500.00

3.1.6. Codificación en lenguaje C

trabajo_autonomo > C ejercicio_96.c > main()

```c
#include <stdio.h>

void main() {
    //declaracion de datos
    float DOSIS_ADULTO = 500.0;
    float DOSIS_NINO = 250.0;
    int EDAD_ADULTO = 18;
    float PESO_MIN = 5.0;
    float peso = 0.0;
    int edad = 0;
    float dosis = 0.0;
    float valor_min = 0.0;
    //entrada
    printf("ingrese su peso en kg\n");
    scanf("%f", &peso);
    printf("ingrese su edad\n");
    scanf("%d", &edad);
    //proceso
    dosis = (edad >= EDAD_ADULTO ? DOSIS_ADULTO * (peso / 70) : DOSIS_NINO * (peso / 30));
    //evaluación y salida
    if (peso < edad && peso < dosis) {
    valor_min = peso;
    }
    if (edad < peso && edad < dosis) {
    valor_min = edad;
    }
    if (dosis < peso && dosis < edad) {
    valor_min = dosis;
    }
    printf("el valor minimo de todas las variables es: %.2f\n", valor_min);

    if ((edad >= EDAD_ADULTO) && (peso >= PESO_MIN)) {
        printf("usted es mayor de edad y su dosis necesaria es: %.2f\n", dosis);
    } else {
        printf("usted es menor de edad y su dosis necesaria es: %.2f\n", dosis);
    }
}
```

3.1.7. Salida del programa

```
C:\Users\MSI\Desktop\TDLP\teoria_de_la_programacion\PracticasC\trabajo_autonomo>gcc ejercicio_96.c -o ejercicio_96

C:\Users\MSI\Desktop\TDLP\teoria_de_la_programacion\PracticasC\trabajo_autonomo>ejercicio_96.exe
ingrese su peso en kg
70.2
ingrese su edad
18
el valor minimo de todas las variables es: 18.00
usted es mayor de edad y su dosis necesaria es: 501.43
```

3.2.1. Enunciado del algoritmo

EJ-097

En una empresa de transporte, cree un algoritmo que calcule el costo de estacionamiento. Lea las horas de uso (real), el tipo de vehiculo (caracter: 'A' auto, 'M' moto, 'C' camion) y la modalidad (caracter: 'H' por hora, 'D' dia completo). Use constantes simbolicas TARIFA_A = 2.0, TARIFA_M = 1.0, TARIFA_C = 4.0, MAX_HORAS = 24. Evalúe las expresiones logicas

para validar (horas > 0 AND horas <= MAX_HORAS). Calcule el costo con expresiones aritmeticas. Asigne el resultado a costo. Muestre por pantalla el costo. Ademas, calcule el valor minimo de todas las variables utilizadas. Asegurese de inicializar todas las variables antes de usarlas.

3.2.2. Análisis del problema

Datos de entrada

•horas de uso
•tipo de vehículo
•modalidad

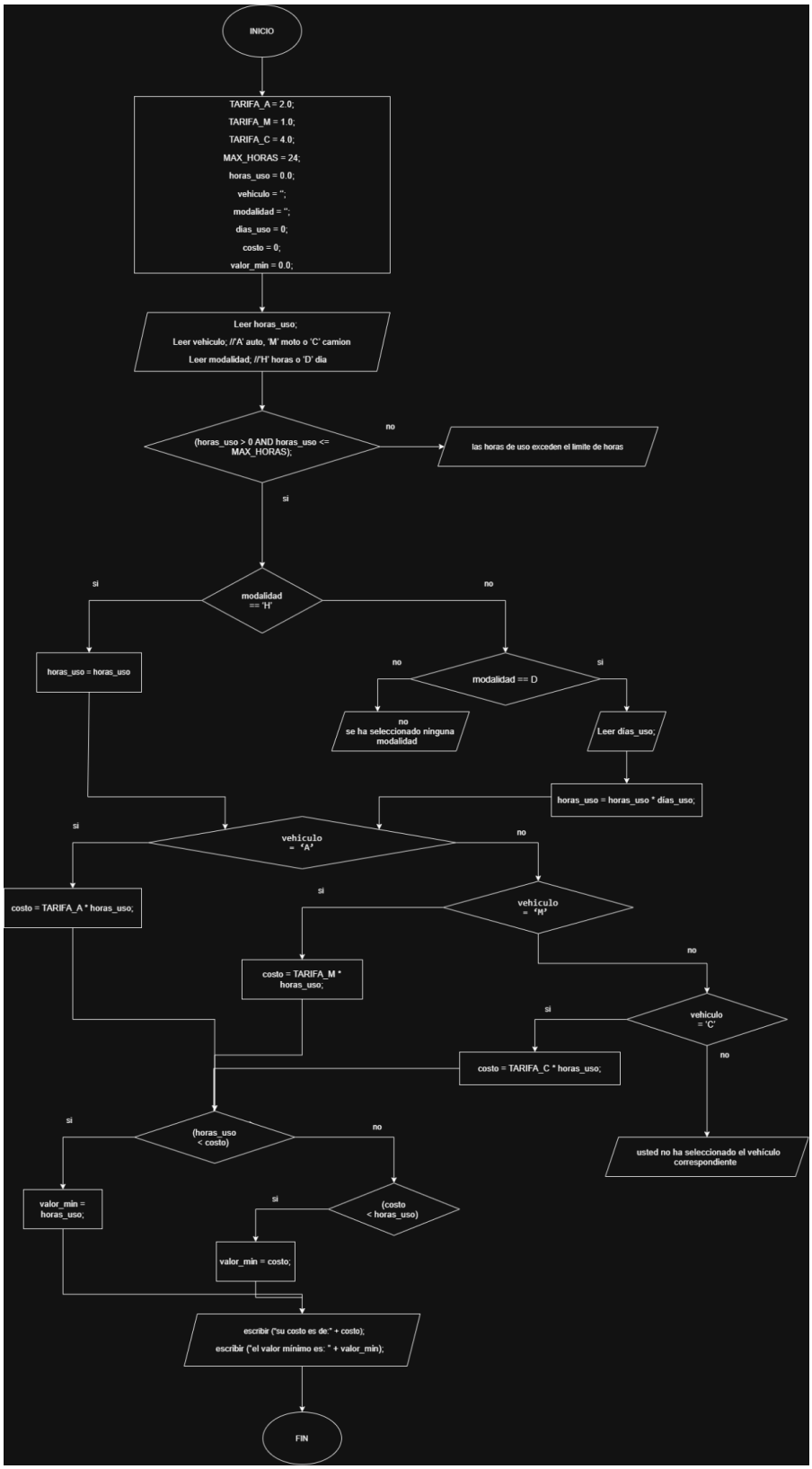
Proceso

1. declarar como constantes TARIFA_A = 2.0, TARIFA_M = 1.0, TARIFA_C = 4.0, MAX_HORAS = 24.
2. Leer horas de uso, tipo de vehiculo (A, M o C) y modalidad (H o D), por el teclado.
3. Evaluar (horas > 0 AND horas <= MAX_HORAS)
4. Calcular el costo y asignarle el valor a costo.
5. Calcular el valor mínimo de todas las variables que se usaron

Salida

Costo

3.2.3. Diagrama de flujo



3.2.4. Pseudocódigo

```
INICIO
//declaración de datos
TARIFA_A = 2.0;
TARIFA_M = 1.0;
TARIFA_C = 4.0;
MAX_HORAS = 24;
horas_uso = 0.0;
vehiculo = ‘’;
modalidad = ‘’;
dias_uso = 0;
costo = 0.0;
valor_min = 0.0;
//entrada
Leer horas_uso;
Leer vehiculo; //‘A’ auto, ‘M’ moto o ‘C’ camion
Leer modalidad; //‘H’ horas o ‘D’ dia
//proceso
Si (horas_uso > 0 AND horas_uso <= MAX_HORAS) entonces
Sino (“las horas de uso exceden el límite de horas”);
Si modalidad == ‘H’ entonces
      horas_uso = horas_uso
si modalidad == ‘D’ entonces
      Leer días_uso;
      horas_uso = horas_uso * días_uso;
Sino escribir (“no se ha seleccionado ninguna modalidad”);
Si vehiculo = ‘A’ entonces
      costo = TARIFA_A * horas_uso;
Sino si vehiculo = ‘M’ entonces
      costo = TARIFA_M * horas_uso;
sino si vehiculo = ‘C’ entonces
      costo = TARIFA_C * horas_uso;
sino escribir (“usted no ha seleccionado el vehículo correspondiente”)
si (horas_uso < costo) entonces
      valor_min = horas_uso;
si (costo < horas_uso) entonces
      valor_min =  costo;

      escribir (“su costo es de:” + costo);
      escribir (“el valor mínimo es: ” + valor_min);
FIN
```

3.2.5. Prueba de escritorio

Pasos	1	2	3	4	5	6	7	8
Caso	Horas de uso	Vehículo	Modalidad	Días de uso	costo	Valor minimo	¿Las horas son validas?	Salida
Caso 1	15	A	H	----	30	15	verdadero	Su costo es: 30.00 Valor mínimo = 15.00
Caso 2	21	C	D	2	168.00	42.00	verdadero	Su costo es: 168.00 Valor mínimo = 42.00

3.2.6. Codificación en lenguaje C

```c
#include <stdio.h>

void main() {
    //declaracion de variables
    float TARIFA_A = 2.0;
    float TARIFA_M = 1.0;
    float TARIFA_C = 4.0;
    float MAX_HORAS = 24;
    float horas_uso = 0.0;
    char vehiculo;
    char modalidad;
    int dias_uso = 0;
    float costo = 0.0;
    float valor_min = 0.0;
    //entrada
    printf("ingrese las horas de uso:\n");
    scanf("%f", &horas_uso);
    printf("ingrese el tipo de vehiculo (A) auto, (M) moto y (C) camion:\n");
    scanf(" %c", &vehiculo);
    printf("ingrese la modalidad de uso (H) horas o (D) dias:\n");
    scanf(" %c", &modalidad);
    //proceso y evaluacion
    if (horas_uso > 0 && horas_uso <= MAX_HORAS) {
    } else {
        printf("las horas de uso exceden el limite de horas\n");
    }
    if (modalidad == 'H' || modalidad == 'h') {
        horas_uso = horas_uso;
    } else if (modalidad == 'D' || modalidad == 'd') {
        printf("ingrese los dias usados:\n");
        scanf("%d", &dias_uso);
        horas_uso = horas_uso * dias_uso;
    } else {
        printf("no ha seleccionado ninguna modalidad\n");
    }
    if (vehiculo == 'A' || vehiculo == 'a') {
        costo = TARIFA_A * horas_uso;
    } else if (vehiculo == 'M' || vehiculo == 'm') {
        costo = TARIFA_M * horas_uso;
    } else if (vehiculo == 'C' || vehiculo == 'c') {
        costo = TARIFA_C * horas_uso;
    } else {
        printf("usted no ha seleccionado el vehículo correspondiente\n");
    } if (horas_uso < costo) {
        valor_min = horas_uso;
    } if (costo < horas_uso) {
        valor_min = costo;
    } if (costo == horas_uso){
        valor_min = horas_uso;
    }
    printf("su costo es: %.2f\n", costo);
    printf("el valor minimo es: %.2f\n", valor_min);
}
```

3.2.7. Salida del programa

```
C:\Users\MSI\Desktop\TDLP\teoria_de_la_programacion\PracticasC\trabajo_autonomo>gcc ejercicio_97.c -o ejercicio_97

C:\Users\MSI\Desktop\TDLP\teoria_de_la_programacion\PracticasC\trabajo_autonomo>ejercicio_97.exe
ingrese las horas de uso:
15
ingrese el tipo de vehiculo (A) auto, (M) moto y (C) camion:
a
ingrese la modalidad de uso (H) horas o (D) dias:
h
su costo es: 30.00
el valor minimo es: 15.00
```

3.3.1. Enunciado del algoritmo

EJ-098

En un cine, diseñe un algoritmo que calcule los descuentos de nomina. Lea el salario bruto (real), las horas extras (entero) y el numero de hijos (entero). Use constantes simbolicas ISSS = 0.03, AFP = 0.0725, RENTA = 0.10, ASIGNACION_HIJO = 30.0, PAGO_HORA_EXTRA = 1.5. Calcule: descuentos = salario_bruto * (ISSS + AFP + RENTA); asignaciones = hijos * ASIGNACION_HIJO; pago_extra = horas_extra * salario_bruto / 240 * PAGO_HORA_EXTRA; salario_neto = salario_bruto + pago_extra + asignaciones - descuentos. Muestre por pantalla el desglose completo. Ademas, calcule el valor minimo de todas las variables utilizadas. Incluya una variable auxiliar para llevar un contador de operaciones.

3.3.2. Análisis del problema

Datos de entrada

- Salario bruto (real)
- Horas extra (entero)
- Número de hijos (entero)

Proceso

1. Declarar variables ISSS = 0.03, AFP = 0.0725, RENTA = 0.10, ASIGNACION_HIJO = 30.0, PAGO_HORA_EXTRA = 1.5, salario bruto, descuentos, asignaciones, pago_extra y salario_neto como real, horas extra, numero de hijos y contador de operaciones como entero.
2. Leer salario bruto, horas extra y numero de hijos desde el teclado.
3. Calcular descuentos = salario_bruto * (ISSS + AFP + RENTA)
4. Calcular asignaciones = hijos * ASIGNACION_HIJO
5. Calcular pago_extra = horas_extra * salario_bruto / 240 * PAGO_HORA_EXTRA
6. Calcular salario_neto = salario_bruto + pago_extra + asignaciones - descuentos.
7. Calcular valor minimo

Salida

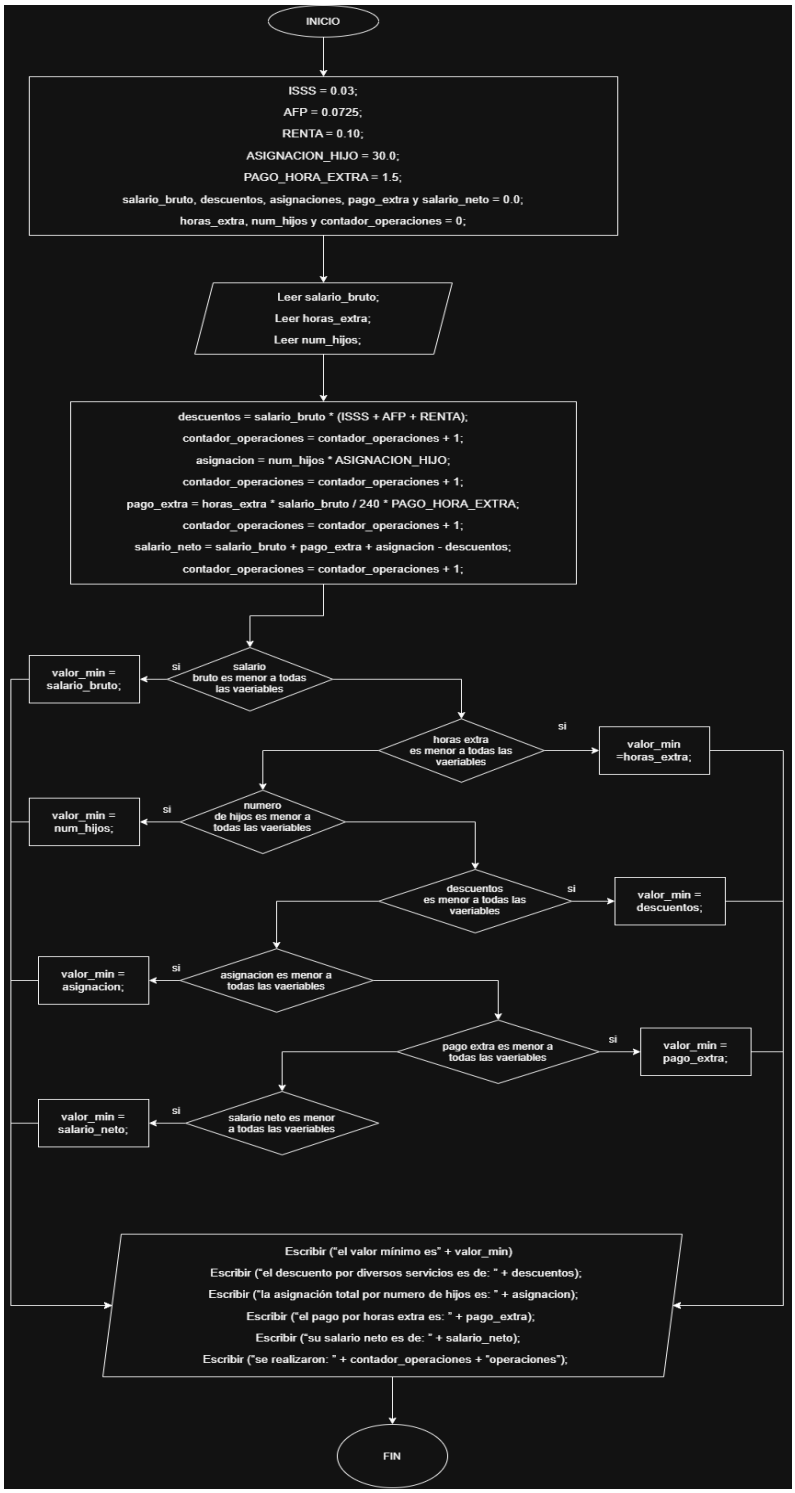
Descuento

Asignaciones

Pago extra

Salario neto

3.3.3. Diagrama de flujo



3.3.4. Pseudocódigo

```
INICIO
//declaración de datos
ISSS = 0.03;
AFP = 0.0725;
RENTA = 0.10;
ASIGNACION_HIJO = 30.0;
PAGO_HORA_EXTRA = 1.5;
salario_bruto, descuentos, asignaciones, pago_extra y salario_neto = 0.0;
horas_extra, num_hijos y contador_operaciones = 0;
//entrada
Leer salario_bruto;
Leer horas_extra;
Leer num_hijos;
//proceso
      descuentos = salario_bruto * (ISSS + AFP + RENTA);
      contador_operaciones = contador_operaciones + 1;
      asignacion = num_hijos * ASIGNACION_HIJO;
      contador_operaciones = contador_operaciones + 1;
      pago_extra = horas_extra * salario_bruto / 240 * PAGO_HORA_EXTRA;
      contador_operaciones = contador_operaciones + 1;
      salario_neto = salario_bruto + pago_extra + asignacion - descuentos;
      contador_operaciones = contador_operaciones + 1;
//evaluación
Si (salario_bruto < horas_extra && salario_bruto < num_hijos && salario_bruto
< descuentos && salario_bruto < asignacion && salario_bruto < pago_extra &&
salario_bruto < salario_neto) entonces
      valor_min = salario_bruto;
Si (horas_extra < salario_bruto && horas_extra < num_hijos && horas_extra <
descuentos && horas_extra < asignacion && horas_extra < pago_extra &&
horas_extra < salario_neto) entonces
      valor_min =horas_extra;
Si (num_hijos < salario_bruto && num_hijos < horas_extra && num_hijos <
descuentos && num_hijos < asignacion && num_hijos < pago_extra && num_hijos <
salario_neto) entonces
      valor_min = num_hijos;
Si (descuentos < salario_bruto && descuentos < horas_extra && descuentos <
num_hijos && descuento < asignación && descuentos < pago_extra && descuentos <
salario_neto) entonces
      valor_min = descuentos;
si (asignacion < salario_bruto && asignacion < horas_extra && asignacion <
num_hijos && asignacion < descuento && asignacion < pago_extra && asignacion <
salario_neto) entonces
      valor_min = asignacion;
si (pago_extra < salario_bruto && pago_extra < horas_extra && pago_extra <
num_hijos && pago_extra < descuento && pago_extra < asignacion && pago_extra <
salario_neto) entonces
      valor_min = pago_extra;
si (salario_neto < salario_bruto && salario_neto < horas_extra && salario_neto
< num_hijos && salario_neto < descuento && salario_neto < asignacion &&
salario_neto < pago_extra) entonces
      valor_min = salario_neto;
//salida
      Escribir (“el valor mínimo es” + valor_min)
      Escribir (“el descuento por diversos servicios es de: ” + descuentos);
      Escribir (“la asignación total por numero de hijos es: ” + asignacion);
      Escribir (“el pago por horas extra es: ” + pago_extra);
```

```
        Escribir (“su salario neto es de: ” + salario_neto);
        Escribir (“se realizaron: ” + contador_operaciones + “operaciones”);
FIN
```

3.3.5. Prueba de escritorio

Pasos	1	2	3	4	5	6	7	8	9	10
caso	Salario bruto	Horas extra	hijos	descuento	Asignación	Pago extra	Salario neto	Valor mínimo	operaciones	Salida
Caso 1	450.00	5	2	91.13	60.00	6.25	425.13	2.00	4	Valor mínimo: 2.00 Descuento: 91.13 Asignación: 60.00 Pago extra: 6.25 Salario neto: 425.13 Operaciones: 4
Caso 2	521.00	3	0	105.50	0.00	4.34	419.84	0.00	4	Valor mínimo: 0.00 Descuento: 105.50 Asignación: 0.00 Pago extra: 4.34 Salario neto: 419.84 Operaciones: 4

3.3.6. Codificación en lenguaje C

```c
#include <stdio.h>

void main() {
    //declaracion de datos
    float ISSS = 0.03;
    float AFP = 0.0725;
    float RENTA = 0.10;
    float ASIGNACION_HIJO = 30.0;
    float PAGO_HORA_EXTRA = 1.5;
    float salario_bruto, descuento, asignacion, pago_extra, salario_neto, valor_min = 0.0;
    int horas_extra, num_hijos, contador_operaciones = 0;
    //entrada
    printf("ingrese salario bruto \n");
    scanf("%f", &salario_bruto);
    printf("ingrese horas extra: \n");
    scanf("%d", &horas_extra);
    printf("ingrese numero de hijos \n");
    scanf("%d", &num_hijos);
    //proceso
    descuento = salario_bruto * (ISSS + AFP + RENTA);
    contador_operaciones = contador_operaciones + 1;
    asignacion = num_hijos * ASIGNACION_HIJO;
    contador_operaciones = contador_operaciones + 1;
    pago_extra = (horas_extra * salario_bruto) / (240 * PAGO_HORA_EXTRA);
    contador_operaciones = contador_operaciones + 1;
    salario_neto = salario_bruto + pago_extra + asignacion - descuento;
    contador_operaciones = contador_operaciones + 1;
    //evaluacion
    if (salario_bruto < horas_extra && salario_bruto < num_hijos && salario_bruto < descuento && salario_bruto < asignacion && salario_bruto < pago_extra && salario_brut
        valor_min = salario_bruto;
    }
    if (horas_extra < salario_bruto && horas_extra < num_hijos && horas_extra < descuento && horas_extra < asignacion && horas_extra < pago_extra && horas_extra < salari
        valor_min =horas_extra;
    }
    if (num_hijos < salario_bruto && num_hijos < horas_extra && num_hijos < descuento && num_hijos < asignacion && num_hijos < pago_extra && num_hijos < salario_neto) {
        valor_min = num_hijos;
    }
    if (descuento < salario_bruto && descuento < horas_extra && descuento < num_hijos && descuento < asignacion && descuento < pago_extra && descuento < salario_neto) {
        valor_min = descuento;
    }
    if (asignacion < salario_bruto && asignacion < horas_extra && asignacion < num_hijos && asignacion < descuento && asignacion < pago_extra && asignacion < salario_net
        valor_min = asignacion;
    }
    if (pago_extra < salario_bruto && pago_extra < horas_extra && pago_extra < num_hijos && pago_extra < descuento && pago_extra < asignacion && pago_extra < salario_net
        valor_min = pago_extra;
    }
    if (salario_neto < salario_bruto && salario_neto < horas_extra && salario_neto < num_hijos && salario_neto < descuento && salario_neto < asignacion && salario_neto <
        valor_min = salario_neto;
    }
    //salida
    printf("el descuento por diversos servicios es: %.2f\n", descuento);
    printf("la asignacion total por numero de hijos es: %.2f\n", asignacion);
    printf("el pago por horas extras es: %.2f\n", pago_extra);
    printf("su salario neto es: %.2f\n", salario_neto);
    printf("se realizaron %d operaciones\n", contador_operaciones);
    printf("el valor minimo es: %.2f\n", valor_min);
}
```

3.3.7. Salida del programa

```
C:\Users\MSI\Desktop\TDLP\teoria_de_la_programacion\PracticasC\trabajo_autonomo>gcc ejercicio_98.c -o ejercicio_98

C:\Users\MSI\Desktop\TDLP\teoria_de_la_programacion\PracticasC\trabajo_autonomo>ejercicio_98.exe
ingrese salario bruto
450
ingrese horas extra:
5
ingrese numero de hijos
2
el descuento por diversos servicios es: 91.13
la asignacion total por numero de hijos es: 60.00
el pago por horas extras es: 6.25
su salario neto es: 425.13
se realizaron 4 operaciones
el valor minimo es: 2.00
```

3.4.1. Enunciado del algoritmo

EJ-099

En una zapateria, construya un algoritmo que determine si un estudiante es elegible para una beca. Lea el promedio academico (real), el ingreso familiar mensual (real) y el numero de hermanos estudiando (entero). Use constantes simbolicas PROMEDIO_MIN = 8.5, INGRESO_MAX = 1500.0, HERMANOS_MIN = 2. Evalúe las expresiones logicas compuestas: (promedio >= PROMEDIO_MIN AND ingreso <= INGRESO_MAX) OR (promedio >= PROMEDIO_MIN AND hermanos >= HERMANOS_MIN). Asigne el resultado a una variable beca_aprobada. Muestre por pantalla la decision. Ademas, calcule el valor minimo de todas las variables utilizadas. Agregue al final un resumen de los datos ingresados.

3.4.2. Análisis del problema

Datos de entrada

- Promedio académico
- Ingreso

Proceso

familiar mensual

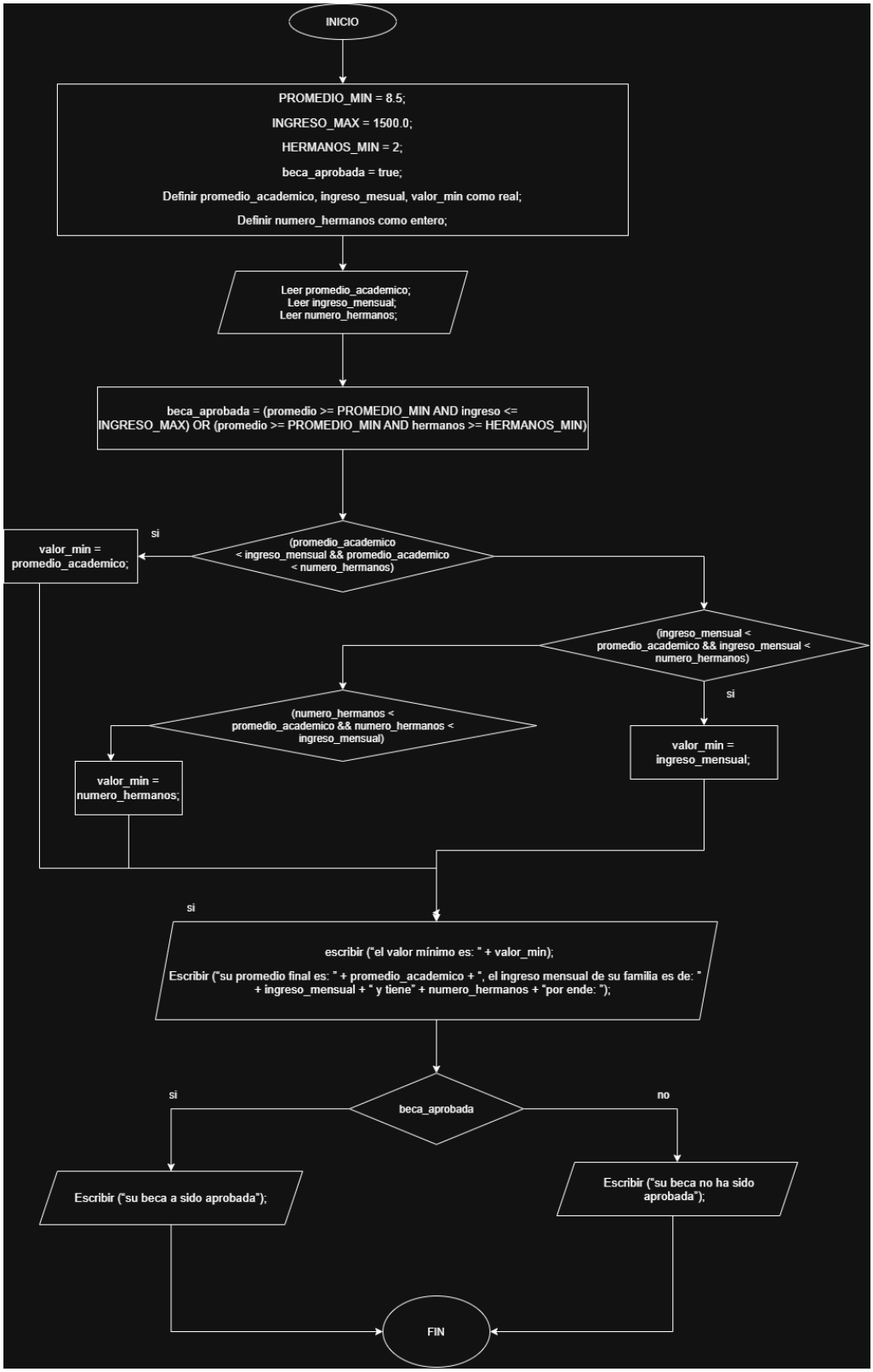
- Numero de hermanos estudiando

1. Declarar PROMEDIO_MIN = 8.5, INGRESO_MAX = 1500.0, HERMANOS_MIN = 2, promedio académico e ingreso familiar mensual como real y numero de hermanos estudiando como entero.
2. Calcular el valor minimo de todas las variables que se usaron
3. Evaluar (promedio >= PROMEDIO_MIN AND ingreso <= INGRESO_MAX) OR (promedio >= PROMEDIO_MIN AND hermanos >= HERMANOS_MIN) y asigarle el valor de beca_aprobada.

Salida

Mostrar valor minimo promedio académico, ingreso familiar y numero de hermanos
Mostrar si aprueba la beca o si no lo hizo.

3.4.3. Diagrama de flujo



3.4.4. Pseudocódigo

```
INICIO
//declaración de datos
PROMEDIO_MIN = 8.5;
INGRESO_MAX = 1500.0;
HERMANOS_MIN = 2;
beca_aprobada = true;
Definir promedio_academico, ingreso_mesual, valor_min como real;
Definir numero_hermanos como entero;
//entrada
Leer promedio_academico;
Leer ingreso_mensual;
Leer numero_hermanos;
//proceso
      beca_aprobada = (promedio >= PROMEDIO_MIN AND ingreso <= INGRESO_MAX)
      OR (promedio >= PROMEDIO_MIN AND hermanos >= HERMANOS_MIN)
//salida y evaluación de condiciones
Si (promedio_academico < ingreso_mensual && promedio_academico <
numero_hermanos) entonces
      valor_min = promedio_academico;
Si (ingreso_mensual < promedio_academico && ingreso_mensual < numero_hermanos)
entonces
      valor_min = ingreso_mensual;
Si (numero_hermanos < promedio_academico && numero_hermanos < ingreso_mensual)
entonces
      valor_min = numero_hermanos;
      escribir ("el valor mínimo es: " + valor_min);
      Escribir ("su promedio final es: " + promedio_academico + ", el ingreso
      mensual de su familia es de: " + ingreso_mensual + " y tiene" +
      numero_hermanos + "por ende: ");
Si beca_aprobada entonces
      Escribir ("su beca a sido aprobada");
Sino
      Escribir ("su beca no ha sido aprobada");
FIN
```

3.4.5. Prueba de escritorio

Pasos	1	2	3	4	5	6
Caso	Promedio académico	Ingreso mensual	Hermanos	Valor mínimo	¿Beca aprobada?	Salida
Caso 1	9.68	560.50	3	3	Verdadero	Su beca a sido aprobada
Caso 2	7.50	582.50	2	2	Falso	Su beca no a sido aprobada

3.4.6. Codificación en lenguaje C

```c
#include <stdio.h>
#include <stdbool.h>

void main() {
    //declaracion de datos
    float PROMEDIO_MIN = 8.5;
    float INGRESO_MAX = 1500.0;
    float HERMANOS_MIN = 2;
    bool beca_aprobada = true;
    float promedio_academico, ingreso_mensual, valor_min = 0.0;
    int numero_hermanos = 0;
    //entrada de datos
    printf("ingrese su promedio academico:\n");
    scanf("%f", &promedio_academico);
    printf("ingrese sus ingresos mensuales:\n");
    scanf("%f", &ingreso_mensual);
    printf("ingrese el numero de hermanos:\n");
    scanf("%d", &numero_hermanos);
    //proceso
    beca_aprobada = (promedio_academico >= PROMEDIO_MIN && ingreso_mensual <= INGRESO_MAX) || (promedio_academico >= PROMEDIO_MIN && numero_hermanos >= HERMANOS_MIN);
    //salida y evaluacion de condiciones
    if (promedio_academico < ingreso_mensual && promedio_academico < numero_hermanos) {
        valor_min = promedio_academico;
    }
    if (ingreso_mensual < promedio_academico && ingreso_mensual < numero_hermanos) {
        valor_min = ingreso_mensual;
    }
    if (numero_hermanos < promedio_academico && numero_hermanos < ingreso_mensual){
        valor_min = numero_hermanos;
    }
    printf("el valor minimo es: %.2f\n", valor_min);
    printf("su promedio final es: %.2f \nel ingreso mensual de su familia es: %.2f y tiene: %d hermanos \npor ende: \n",promedio_academico, ingreso_mensual, numero_herma
    if (beca_aprobada){
        printf("su beca a sido aprobada. \n");
    } else {
        printf("su beca no a sido aprobada. \n");
```

3.4.7. Salida del programa

```
C:\Users\MSI\Desktop\TDLP\teoria_de_la_programacion\PracticasC\trabajo_autonomo>gcc ejercicio_99.c -o ejercicio_99

C:\Users\MSI\Desktop\TDLP\teoria_de_la_programacion\PracticasC\trabajo_autonomo>ejercicio_99.exe
ingrese su promedio academico:
9.68
ingrese sus ingresos mensuales:
560.5
ingrese el numero de hermanos:
3
el valor minimo es: 3.00
su promedio final es: 9.68
el ingreso mensual de su familia es: 560.50 y tiene: 3 hermanos
por ende:
su beca a sido aprobada.
```

3.5.1. Enunciado del algoritmo

EJ-100

En una jugueteria, diseñe un algoritmo que simule un credito hipotecario. Lea el valor de la vivienda (real), el enganche (real), el plazo en años (entero) y la tasa de interes anual (real). Use constantes simbolicas SEGURO_VIDA = 0.003, SEGURO_INMUEBLE = 0.005, COMISION = 0.01, MESES_ANIO = 12. Calcule: monto_credito = valor - enganche; comision = monto_credito * COMISION; cuota_mensual = (monto_credito * (1 + interés anual)) / (plazo * MESES_AÑO); total = cuota_mensual * plazo * MESES_AÑO + comision + monto_credito * (SEGURO_VIDA + SEGURO_INMUEBLE) * plazo. Muestre por pantalla el desglose. Ademas, calcule el valor minimo de todas las variables utilizadas. Asegurese de que las expresiones logicas esten bien parentizadas.

3.5.2. Análisis del problema

Datos de entrada

- Valor de la vivienda
- Valor de enganche
- Plazo en años
- Tasa de interés anual

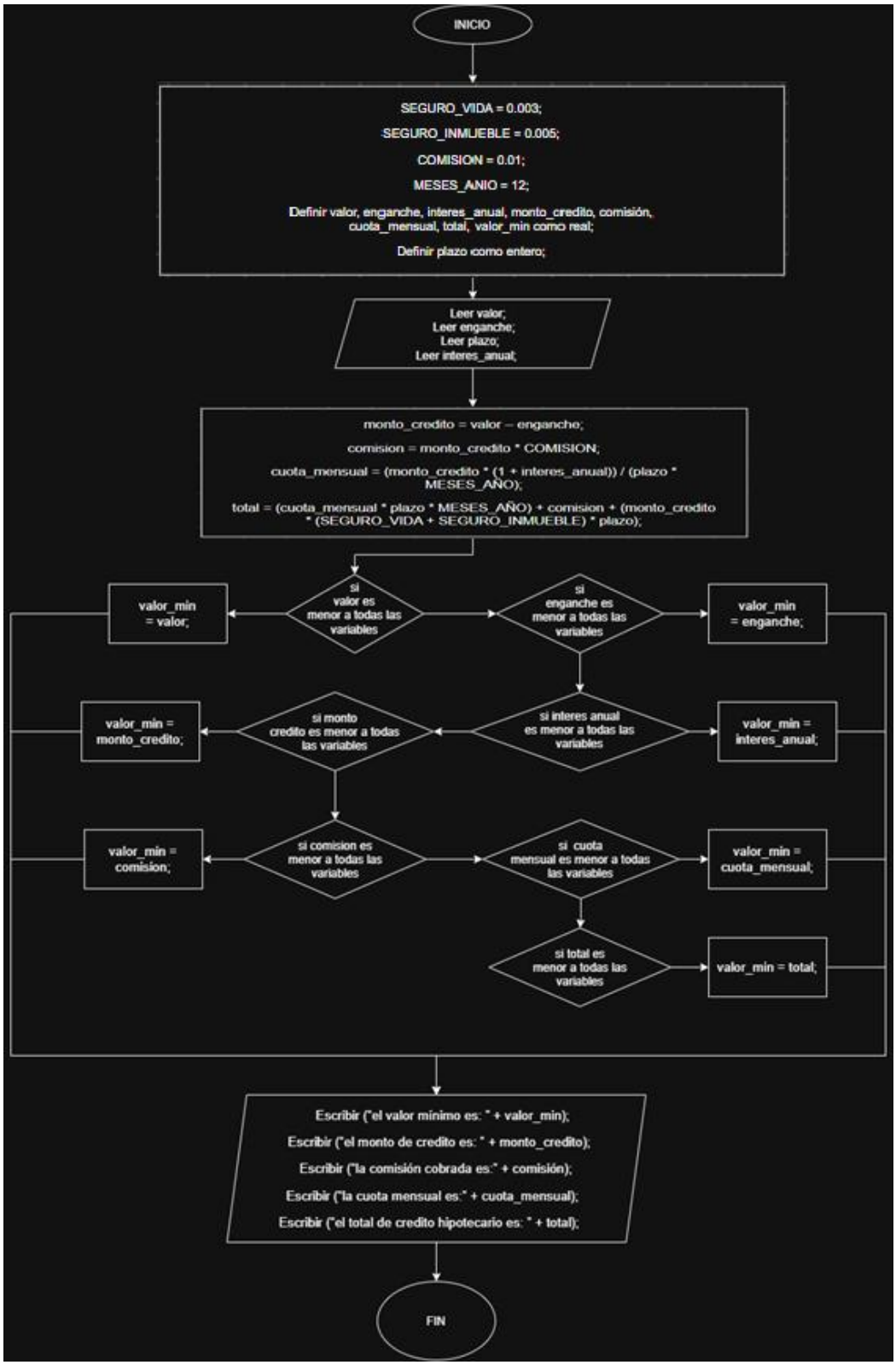
Proceso

1. Declarar constantes SEGURO_VIDA = 0.003, SEGURO_INMUEBLE = 0.005, COMISION = 0.01, MESES_AÑO = 12.
2. Declarar valor de la vivienda, valor de enganche, tasa de interés anual, monto-crédito, comisión, cuota mensual, total como reales y plazo en años como entero.
3. Leer valor de la vivienda, valor de enganche, plazo en años y tasa de interés anual desde el teclado.
4. Calcular monto_credito = valor – enganche
5. Calcular comision = monto_credito * COMISION
6. Calcular cuota_mensual = (monto_credito * (1 + interés anual)) / (plazo * MESES_AÑO)
7. Calcular total = (cuota_mensual * plazo * MESES_AÑO) + comision + (monto_credito * (SEGURO_VIDA + SEGURO_INMUEBLE) * plazo.)
8. Calcular valor mínimo de todas las variables que se usaron

Salida

Mostrar valor minimo
Mostrar monto del crédito, comisión, cuota mensual y total de crédito.

3.5.3. Diagrama de flujo



3.5.4. Pseudocódigo

```
INICIO
//declarar datos
SEGURO_VIDA = 0.003;
SEGURO_INMUEBLE = 0.005;
COMISION = 0.01;
MESES_ANIO = 12;
Definir valor, enganche, interes_anual, monto_credito, comisión,
cuota_mensual, total, valor_min como real;
Definir plazo como entero;
//entrada
Leer valor;
Leer enganche;
Leer plazo;
Leer interes_anual;
//proceso
      monto_credito = valor – enganche;
      comision = monto_credito * COMISION;
      cuota_mensual = (monto_credito * (1 + interes_anual)) / (plazo *
      MESES_AÑO);
      total = (cuota_mensual * plazo * MESES_AÑO) + comision + (monto_credito
      * (SEGURO_VIDA + SEGURO_INMUEBLE) * plazo);
//evaluación
      SI (valor < enganche && valor < interes_anual && valor < monto_credito
&& valor < comision && valor < cuota_mensual && valor < total) ENTONCES
valor_min = valor;
      SI (enganche < valor && enganche < interes_anual && enganche <
monto_credito && enganche < comision && enganche < cuota_mensual && enganche <
total) ENTONCES
valor_min = enganche;
      SI (interes_anual < valor && interes_anual < enganche && interes_anual
< monto_credito && interes_anual < comision && interes_anual < cuota_mensual
&& interes_anual < total) ENTONCES
valor_min = interes_anual;
      SI (monto_credito < valor && monto_credito < enganche && monto_credito
< interes_anual && monto_credito < comision && monto_credito < cuota_mensual
&& monto_credito < total) ENTONCES
valor_min = monto_credito;
      SI (comision < valor && comision < enganche && comision < interes_anual
&& comision < monto_credito && comision < cuota_mensual && comision < total)
ENTONCES
valor_min = comision;
      SI (cuota_mensual < valor && cuota_mensual < enganche && cuota_mensual
< interes_anual && cuota_mensual < monto_credito && cuota_mensual < comision
&& cuota_mensual < total) ENTONCES
valor_min = cuota_mensual;
      SI (total < valor && total < enganche && total < interes_anual && total
< monto_credito && total < comision && total < cuota_mensual) ENTONCES
valor_min = total;
//salida
      Escribir (“el valor mínimo es: ” + valor_min);
      Escribir (“el monto de credito es: ” + monto_credito);
      Escribir (“la comisión cobrada es:” + comisión);
      Escribir (“la cuota mensual es:” + cuota_mensual);
      Escribir (“el total de credito hipotecario es: ” + total);
FIN
```

3.5.5. Prueba de escritorio

Pasos	1	2	3	4	5	6	7	8	9	10
casos	Valor	enganche	Plazo anual	Interes anual	Monto del crédito	Comisión	Cuota mensual	Total de crédito	Valor mínimo	Salida
Caso 1	112700.00	25000	20	0.05	87700.00	877.00	383.69	106994.00	0.05	Valor mínimo: 0.05 Monto de crédito: 87700.00 Comisión: 877.00 Cuota mensual: 383.69 Total de crédito hipotecario: 106994.00
Caso 2	230000.00	46000.00	10	0.06	184000.00	1840.00	1625.33	211600.00	0.06	Valor mínimo: 0.06 Monto de crédito: 184000.00 Comisión: 1840.00 Cuota mensual: 1625.33 Total de crédito hipotecario: 211600.00

3.5.6. Codificación en lenguaje C



```c
#include <stdio.h>

void main() {
    //declaracion de datos
    float SEGURO_VIDA = 0.003;
    float SEGURO_INMUEBLE = 0.005;
    float COMISION = 0.01;
    int MESES_ANIO = 12;
    float valor, enganche, interes_anual, monto_credito, comision, cuota_mensual, total, valor_min = 0.0;
    int plazo = 0;
    //entrada
    printf("ingrese el valor de la vivienda:\n");
    scanf("%f", &valor);
    printf("ingrese el valor del engache:\n");
    scanf("%f", &enganche);
    printf("ingrese el plazo anual:\n");
    scanf("%d", &plazo);
    printf("ingrese la tasa de interes anual:\n");
    scanf("%f", &interes_anual);
    //proceso
    monto_credito = valor - enganche;
    comision = monto_credito * COMISION;
    cuota_mensual = (monto_credito * (1 + interes_anual)) / (plazo * MESES_ANIO);
    total = (cuota_mensual * plazo * MESES_ANIO) + comision + (monto_credito * (SEGURO_VIDA + SEGURO_INMUEBLE) * plazo);
    //evaluacion y salida
    if (valor < enganche && valor < interes_anual && valor < monto_credito && valor < comision && valor < cuota_mensual && valor < total) {
        valor_min = valor;
    } if (valor < enganche && valor < interes_anual && valor < monto_credito && valor < comision && valor < cuota_mensual && valor < total) {
        valor_min = valor;
    } if (enganche < valor && enganche < interes_anual && enganche < monto_credito && enganche < comision && enganche < cuota_mensual && enganche < total) {
        valor_min = enganche;
    } if (interes_anual < valor && interes_anual < enganche && interes_anual < monto_credito && interes_anual < comision && interes_anual < cuota_mensual && interes_anua
        valor_min = interes_anual;
    } if (monto_credito < valor && monto_credito < enganche && monto_credito < interes_anual && monto_credito < comision && monto_credito < cuota_mensual && monto_credit
        valor_min = monto_credito;
    } if (comision < valor && comision < enganche && comision < interes_anual && comision < monto_credito && comision < cuota_mensual && comision < total) {
        valor_min = comision;
    } if (cuota_mensual < valor && cuota_mensual < enganche && cuota_mensual < interes_anual && cuota_mensual < monto_credito && cuota_mensual < comision && cuota_mensua
        valor_min = cuota_mensual;
    } if (total < valor && total < enganche && total < interes_anual && total < monto_credito && total < comision && total < cuota_mensual) {
        valor_min = total;
    }
    printf("el valor mínimo es: %.2f\n", valor_min);
    printf("el monto de credito es: %.2f\n", monto_credito);
    printf("la comision cobrada es: %.2f\n", comision);
    printf("la cuota mensual es: %.2f\n", cuota_mensual);
    printf("el total de credito hipotecario es: %.2f\n", total);
}
```

3.5.7. Salida del programa

```
C:\Users\MSI\Desktop\TDLP\teoria_de_la_programacion\PracticasC\trabajo_autonomo>gcc ejercicio_100.c -o ejercicio_100

C:\Users\MSI\Desktop\TDLP\teoria_de_la_programacion\PracticasC\trabajo_autonomo>ejercicio_100.exe
ingrese el valor de la vivienda:
112700
ingrese el valor del engache:
25000
ingrese el plazo anual:
20
ingrese la tasa de interes anual:
0.05
el valor m├¡nimo es: 0.05
el monto de credito es: 87700.00
la comision cobrada es: 877.00
la cuota mensual es: 383.69
el total de credito hipotecario es: 106994.00
```

Nivel medio

3.6.1. Enunciado del algoritmo

EJ-046

Construya un algoritmo que lea un carácter desde teclado y lo almacene en la variable c. Use una constante literal 'A' como carácter de referencia. Evalúe las expresiones lógicas: (c == 'A'), (c >= 'A' AND c <= 'Z') y (c >= '0' AND c <= '9'). Muestre por pantalla un mensaje indicando si el carácter ingresado es igual al de referencia, si es letra mayúscula o si es dígito. Identifique el tipo de dato de cada variable.

3.6.2. Análisis del problema

Datos de entrada:

- c

Proceso

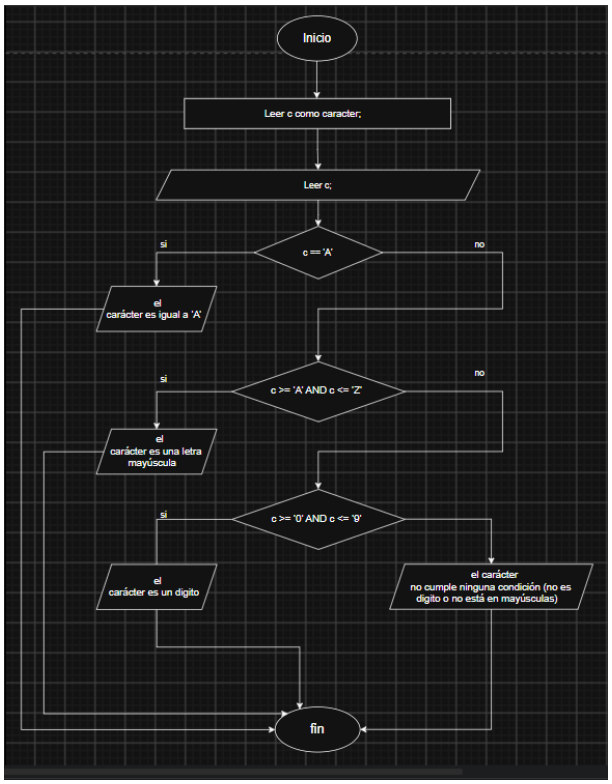
1. Declarar c como carácter
2. Leer c desde el teclado
3. Evaluar la 1era expresión lógica (c == 'A'). si la expresión es verdadera determinar que el carácter es igual al de la referencia
4. Evaluar la 2da expresión lógica (c >= 'A' AND c <= 'Z'). si la expresión es verdadera determinar que el carácter es una letra mayúscula.
5. Evaluar la 3ra expresión lógica (c >= '0' AND c <= '9'). Si la expresión es verdadera determinar que el carácter es un dígito.

Salida

c: es igual al carácter 'A'.
c: es una letra mayúscula.

c: es un dígito.

3.6.3. Diagrama de flujo



3.6.4. Pseudocódigo

```
INICIO
//variable
Leer c como caracter;
//entrada
escribir (“ingrese carácter (c)”);
Leer c;
//evaluación y salida
Si c == ‘A’ entonces;
escribir (“el carácter es igual a ‘A’”);
sino Si c >= ‘A’ AND c <= ‘Z’ entonces;
escribir (“el carácter es una letra mayúscula”);
sino Si c >= ‘0’ AND c <= ‘9’ entonces;
escribir (“el carácter es un digito”);
sino
escribir (“el carácter no cumple ninguna condición, no es digito o está en
mayúscula”);
FIN.
```

3.6.5. Prueba de escritorio

Pasos	1	2	3	4	5
Casos	c	c == ‘A’	c >= ‘A’ AND c <= ‘Z’	c >= ‘0’ AND c <= ‘9’	Salida
Caso 1	D	falso	verdadero	falso	El carácter es una letra mayúscula.
Caso 2	5	falso	falso	verdadero	El carácter es un digito.

3.6.6. Codificación en lenguaje C

ejercicio_46.c ejercicio_47.c

trabajo_autonomo > ejercicio_46.c > main()

```c
#include <stdio.h>

void main(){
    char c = '\0';

    printf("ingrese el valor de c: \n");
    scanf("%c", &c);

    if (c == 'A') {
        printf("el caracter es igual a 'A'\n");
    } else if (c >= 'A' && c <= 'Z') {
        printf("el caracter es una letra mayuscula\n");
    } else if (c >= '0' && c <= '9') {
        printf("el caracter es un numero\n");
    } else {
        printf("el caracter es una letra minuscula\n");
    }
}
```

3.6.7. Salida del programa

```
Microsoft Windows [Versión 10.0.26100.9711]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\MSI\Desktop\TDLP\teoria_de_la_programacion\PracticasC>cd trabajo_autonomo

C:\Users\MSI\Desktop\TDLP\teoria_de_la_programacion\PracticasC\trabajo_autonomo>gcc ejercicio_46.c -o ejercicio_46

C:\Users\MSI\Desktop\TDLP\teoria_de_la_programacion\PracticasC\trabajo_autonomo>ejercicio_46.exe
ingrese el valor de c:
D
el caracter es una letra mayuscula

C:\Users\MSI\Desktop\TDLP\teoria_de_la_programacion\PracticasC\trabajo_autonomo>
```

3.7.1. Enunciado del algoritmo

EJ-047

Diseñe un algoritmo donde se declaren las variables x (entero), z (real), val1 (booleano) y a (carácter). Asigne los literales 25, 3.14, verdadero y 'M' respectivamente. Realice las operaciones secuenciales: asigne a val1 el resultado de la expresión lógica (x > 20 AND z < 4.0), luego asigne a a el carácter 'F' si val1 es verdadero. Muestre por pantalla todos los valores finales de las variables.

3.7.2. Análisis del problema

Datos de entrada:

- x = 25
- z = 3.14
- val1 = verdadero
- a = 'M'

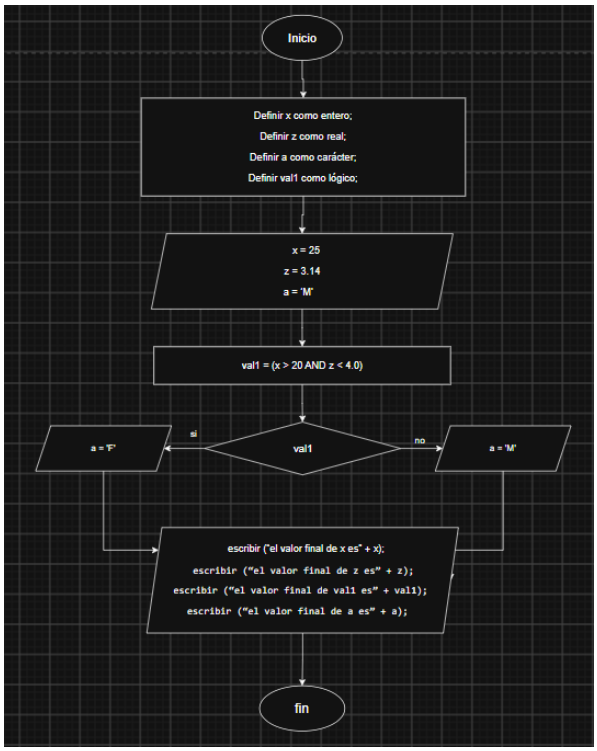
Procedimiento

1. Asignar a val1 el resultado de la expresión lógica (x > 20 AND z < 4.0)
2. Evaluar la expresión lógica (x > 20 AND z < 4.0) si es verdadera determinar a a el carácter 'F'

Salida

resultado final de x
resultado final de z
resultado final de val1
resultado final de a

3.7.3. Diagrama de flujo



3.7.4. Pseudocódigo

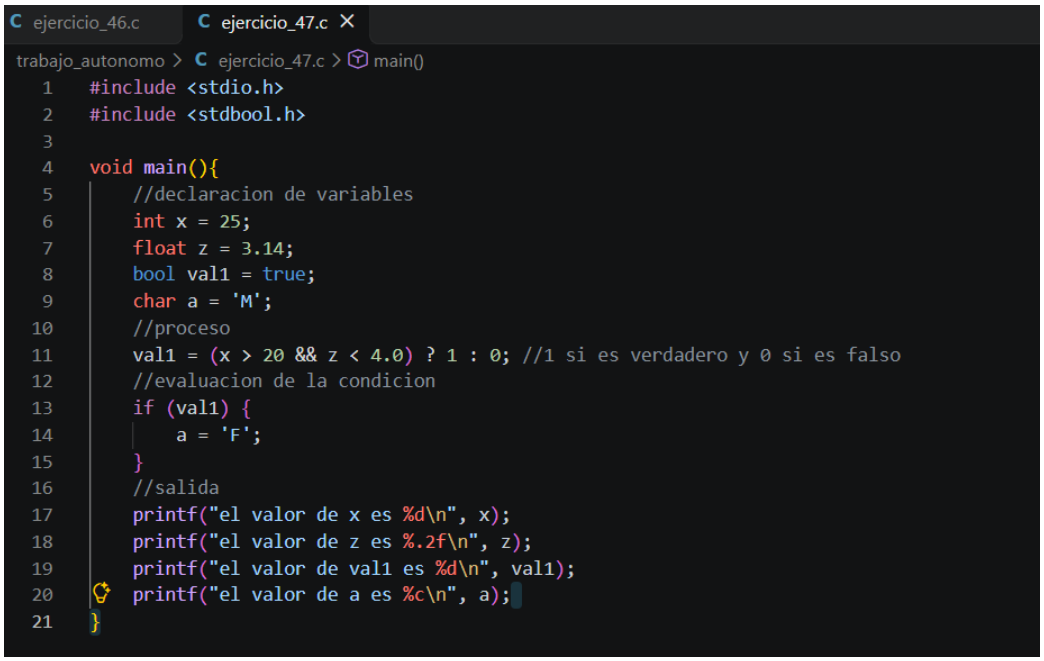
```
INICIO
//definición de variables
Definir x como entero;
Definir z como real;
Definir val1 como lógico;
Definir a como carácter;
```

```
//entrada
x = 25;
z = 3.14;
a = ‘M’;
//proceso
val1 = (x > 20 AND z < 4.0);
//evaluación de la expresión
Si val1 = true entonces;
a = ‘F’;
//salida
escribir (“el valor final de x es” + x);
escribir (“el valor final de z es” + z);
escribir (“el valor final de val1 es” + val1);
escribir (“el valor final de a es” + a);
FIN
```

3.7.5. Prueba de escritorio

Pasos	1	2	3	4	5	6	7	8	9
Caso	x	z	a	`val1 = (x > 20 AND z < 4.0);` `entonces a = F`	x final	z final	a final	Val1	Salida
Caso 1	25	3.14	M	Se cumple	25	3.14	F	verdadero	x = 25 z = 3.14 val1 = 1(true) a = F
Caso 2	10	4.5	G	No se cumple	20	4.5	G	falso	x = 10 z = 4.5 val1 = 0(false) a = G
Nota: el código ya daba los datos de las variables, pero para que haya dos casos cree otro para el resultado falso									

3.7.6. Codificación en lenguaje C



3.7.7. Salida del programa

```
C:\Users\MSI\Desktop\TDLP\teoria_de_la_programacion\PracticasC\trabajo_autonomo>ejercicio_47.exe
el valor de x es 25
el valor de z es 3.14
el valor de val1 es 1
el valor de a es F
```

3.8.1. Enunciado del algoritmo

EJ-048

Cree un algoritmo que solicite al usuario ingresar su nombre (cadena), su año de nacimiento (entero) y el año actual (entero). Use una constante simbólica MAXIMO = 18 para indicar la mayoría de edad. Calcule la edad con la expresión aritmética (año_actual - año_nacimiento). Evalúe la expresión relacional (edad >= MAXIMO) y muestre: "X, tienes Y años y eres mayor de edad" o "X, tienes Y años y eres menor de edad".

3.8.2. Análisis del problema

Datos de entrada:

- Nombre del usuario
- Año de nacimiento
- Año actual

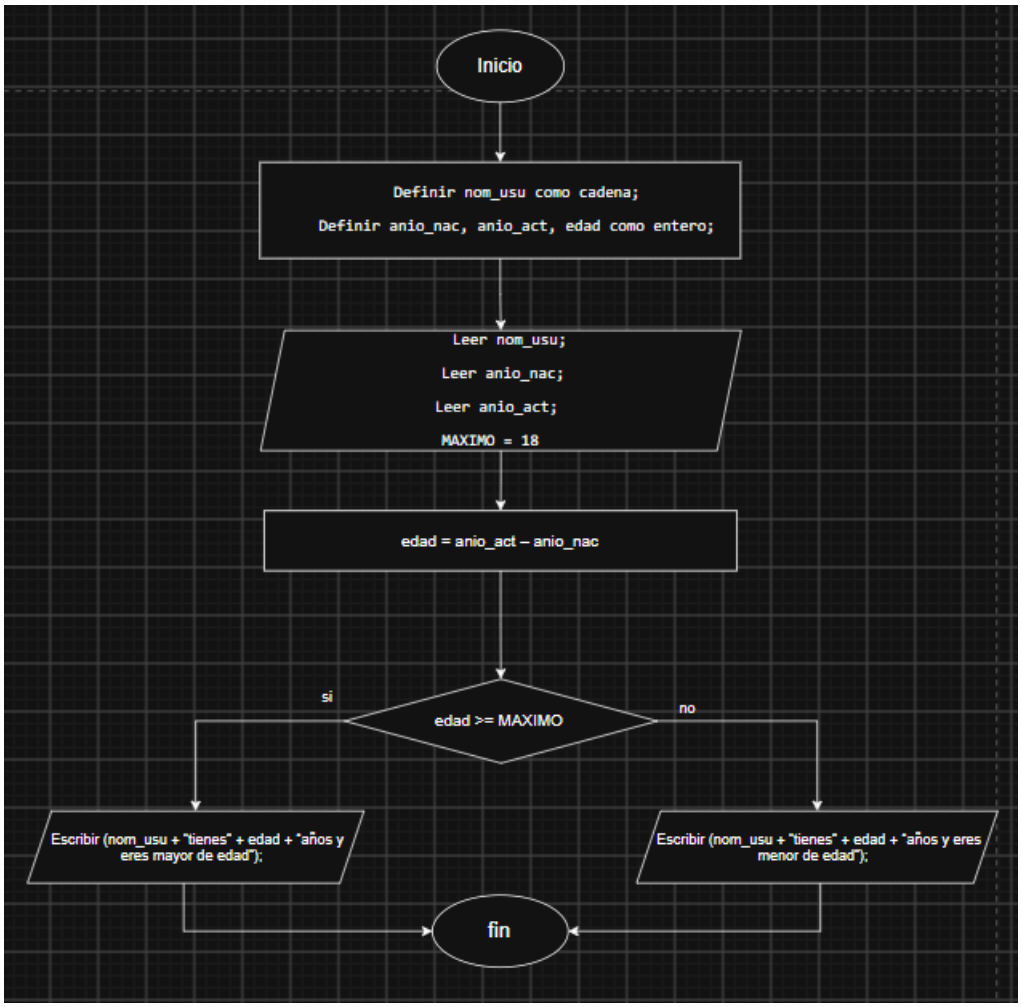
Proceso

1. declarar las variables: nom_usu como cadena, anio_nac, anio_act y edad como número entero.
2. Leer las variables desde el teclado
3. usar constante simbólica MAXIMO = 18 para indicar la mayoría de edad
4. calcular la edad del usuario con la expresión aritmética (anio_act-anio_nac).
5. evaluar la expresión relacional (edad >= MAXIMO).

Salida

nombre del usuario, su edad y si es mayor de edad
nombre del usuario, su edad y si es menor de edad.

3.8.3. Diagrama de flujo



3.8.4. Pseudocódigo

```
INICIO
//definicion de variables
MAXIMO = 18;
Definir nom_usu como cadena;
Definir anio_nac, anio_act, edad como entero;
//variables
Leer nom_usu;
Leer anio_nac;
Leer anio_act;
//proceso
edad = anio_act – anio_nac
//evaluación de la edad
Si edad >= MAXIMO entonces
Escribir (nom_usu + “tienes” + edad + “años y eres mayor de edad”);
Sino
Escribir (nom_usu + “tienes” + edad + “años y eres menor de edad”);
FIN
```

3.8.5. Prueba de escritorio

Pasos	1	2	3	4	5	6	7
Caso	MAXIMO	Usuario	Año de nacimiento	Año actual	Edad	Edad >= MAXIMO?	Salida
Caso 1	18	Fabian	2008	2026	18	Mayor de edad	Fabian tienes 18 años y eres mayor de edad
Caso 2	18	Israel	2012	2026	14	Menor de edad	Israel tienes 14 años y eres menor de edad

3.8.6. Codificación en lenguaje C

trabajo_autonomo > C ejercicio_48.c > main()

```c
#include <stdio.h>
#include <stdbool.h>

void main() {
    int MAXIMO = 18;
    char nom_usu[30];
    int anio_act = 0;
    int anio_nac = 0;
    int edad = 0;

    printf("ingrese su anio de nacimiento: \n");
    scanf("%d", &anio_nac);

    printf("ingrese el anio actual: \n");
    scanf("%d", &anio_act);

    printf("ingrese su nombre de usuario: \n");
    scanf("%29s", nom_usu);

    edad = anio_act - anio_nac;

    if (edad >= MAXIMO) {
        printf(" %s, tienes %d anios y eres mayor de edad.\n", nom_usu, edad);
    } else {
        printf("%s, tienes %d anios y no eres menor de edad.\n", nom_usu, edad);
    }
}
```

3.8.7. Salida del programa

Problemas Salida Consola de depuración **Terminal** Puertos

```
C:\Users\MSI\Desktop\TDLP\teoria_de_la_programacion\PracticasC\trabajo_autonomo>gcc ejercicio_48.c -o ejercicio_48

C:\Users\MSI\Desktop\TDLP\teoria_de_la_programacion\PracticasC\trabajo_autonomo>ejercicio_48.exe
ingrese su anio de nacimiento:
2008
ingrese el anio actual:
2026
ingrese su nombre de usuario:
fabian gordillo
 fabian, tienes 18 anios y eres mayor de edad.
```

3.9.1. Enunciado del algoritmo

EJ-049

Declare una variable entera num2 y una constante simbólica LIMITE = 2. Lea num2 desde teclado. Calcule la expresión aritmética (num2 % LIMITE) y asígnela a residuo. Evalúe la expresión lógica (residuo == 0 AND num2 > 0) y muestre por pantalla el mensaje correspondiente. Identifique claramente cuál es el dato de entrada y cuál es la información procesada

3.9.2. Análisis del problema

Datos de entrada:

- LIMITE = 2
- Variable num2

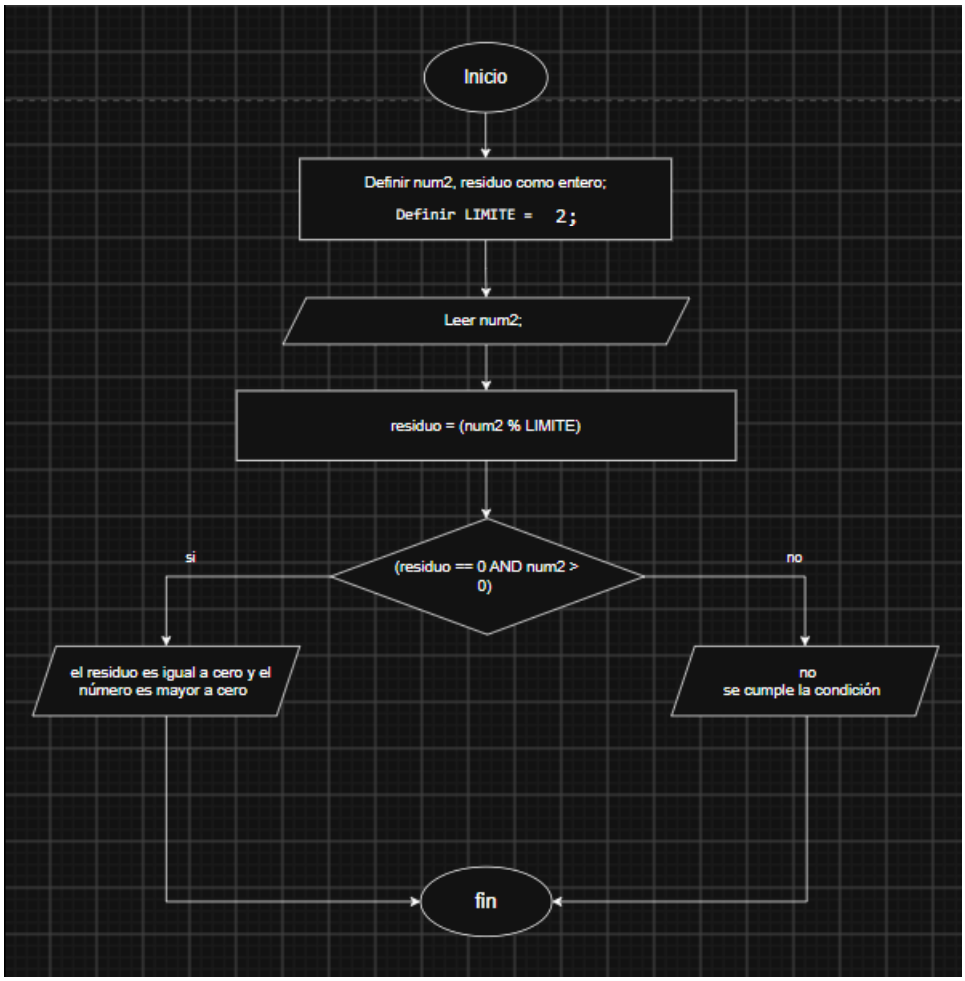
Proceso

1. Declarar la variable num2 como entera, la variable residuo como entero
2. Leer num2 desde el teclado
3. Calcular la expresión aritmética (num2 % LIMITE) y asignarle el nombre de residuo
4. Evaluar la expresión lógica (residuo == 0 AND num2 > 0), si es verdadera establecer el mensaje correspondiente, sino establecer lo contrario.

Salida

La expresión es verdadera
La expresión es falsa.

3.9.3. Diagrama de flujo



3.9.4. Pseudocódigo

```
INICIO
//declaración de variables y constantes
Definir num2, residuo como entero;
Definir LIMITE = 2;
// entrada
Leer num2;
//proceso
residuo = (num2 % LIMITE)
//evaluación de la expresión
Si (residuo == 0 AND num2 > 0) entonces;
Escribir (“Se cumple la condición el residuo es 0 y el número seleccionado es
mayor a cero.”)
Sino
Escribir (“no se cumple la condición”)
FIN
```

3.9.5. Prueba de escritorio

Pasos	1	2	3	4	5
Caso	LIMITE	num2	residuo	`residuo == 0 AND num2 > 0`	`Salida`
Caso 1	2	20	0	si cumple	Se cumple la condición el residuo es 0 y el número seleccionado es mayor a cero.
Caso 2	2	5	1	No se cumple	

3.9.6. Codificación en lenguaje C

```c
C ejercicio_49.c > main()
#include <stdio.h>

int main() {
    int LIMITE = 2;
    int num2 = 0;
    int residuo = 0;

    printf("ingrese numero entero: \n");
    scanf("%d", &num2);

    residuo = num2 % LIMITE;

    if (residuo == 0 && num2 > 0) {
        printf("Se cumple la condición el residuo es 0 y el número seleccionado es mayor a cero.");
    } else {
        printf("no se cumple la condicion \n");
    }
    return 0;
}
```

3.9.7. Salida del programa

```
C:\Users\MSI\Desktop\TDLP\teoria_de_la_programacion\unidad_1\AA\trabajo_autonomo>gcc ejercicio_49.c -o ejercicio_49

C:\Users\MSI\Desktop\TDLP\teoria_de_la_programacion\unidad_1\AA\trabajo_autonomo>ejercicio_49.exe
ingrese numero entero:
20
Se cumple la condici├│n el residuo es 0 y el n├║mero seleccionado es mayor a cero.
C:\Users\MSI\Desktop\TDLP\teoria_de_la_programacion\unidad_1\AA\trabajo_autonomo>
```

3.10.1. Enunciado del algoritmo

EJ-050

Construya un algoritmo que lea tres números enteros val2, a y x por teclado. Use una constante literal 3 para calcular el promedio. Asigne el resultado de la expresión aritmética ((val2 + a + x) / 3) a la variable promedio. Evalúe las expresiones relacionales (val2 > promedio), (a > promedio) y (x > promedio). Muestre por pantalla cuáles de los tres valores están por encima del promedio.

3.10.2. Análisis del problema

Datos de entrada

- Variable val2
- Variable a
- Variable x

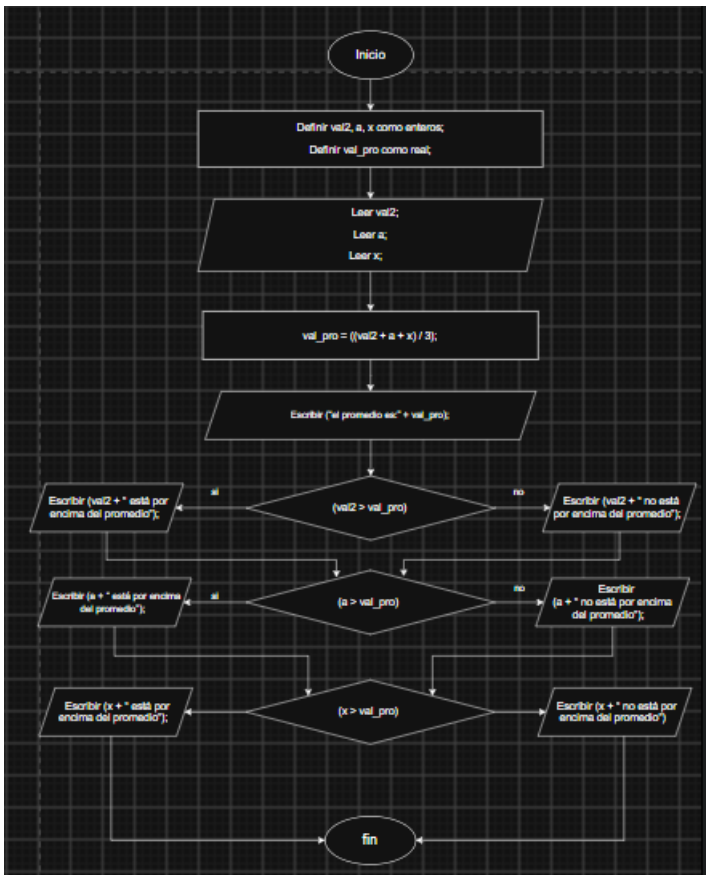
Proceso

1. Declarar las variables val2, a y x como enteros.
2. Leer las variables val2, a y x por el teclado.
3. Asignar a la expresión aritmética ((val2 + a + x) / 3) como variable promedio
4. Evaluar la 1era expresión relacional (val2 > promedio).
5. Evaluar la 2da expresión relacional (a > promedio).
6. Evaluar la 3era expresión relacional (x > promedio).

Salida

val2 es mayor o menor al promedio.
a es mayor o menor al promedio.
x es mayor o menor al promedio.

3.10.3. Diagrama de flujo



3.10.4. Pseudocódigo

```
INICIO
//declaración de variables y constante
Definir val2, a, x como enteros;
Definir val_pro como real;
//entrada
Leer val2;
Leer a;
Leer x;
//proceso
val_pro = ((val2 + a + x) / 3);
//evaluación y salida
Escribir (“el promedio es:” + val_pro);
Si (val2 > val_pro) entonces
Escribir (val2 + “ está por encima del promedio”);
Sino
Escribir (val2 + “ no está por encima del promedio”);
Si (a > val_pro) entonces
Escribir (a + “ está por encima del promedio”);
Sino
Escribir (a + “ no está por encima del promedio”);
Si (x > val_pro) entonces
Escribir (x + “ está por encima del promedio”);
Sino
Escribir (x + “ no está por encima del promedio”)
FIN
```

3.10.5. Prueba de escritorio

Pasos	1	2	3	4	5	6	7	8
Caso	val2	a	x	val_pro	val2 > val_pro	a > val_pro	x > val_pro	Salida
Caso 1	20	45	31	32	falso	verdadero	falso	20 está por debajo del promedio 45 está por encima del promedio 32 está por debajo del promedio
Caso 2	40	21	65	42	falso	falso	verdadero	40 está por debajo del promedio 21 está por debajo del promedio 65 está por encima del promedio

3.10.6. Codificación en lenguaje C

```c
#include <stdio.h>

int main() {
    // Declaración de variables
    int val2, a, x = 0;
    float val_pro = 0.0;
    //entrada
    printf("ingrese un numero entero: \n");
    scanf("%d", &val2);

    printf("ingrese un numero entero: \n");
    scanf("%d", &a);

    printf("ingrese un numero entero: \n");
    scanf("%d", &x);
    //proceso
     val_pro = ((val2 + a + x) / 3);
    //salida y evaluacion
     printf("el valor del promedio es: %.2f \n", val_pro);

     if (val2 > val_pro) {
        printf("el numero %d esta por encima del promedio \n", val2);
    } else {
        printf("el numero %d esta por debajo del promedio \n", val2);
    }

    if (a > val_pro) {
        printf("el numero %d esta por encima del promedio \n", a);
    } else {
        printf("el numero %d esta por debajo del promedio \n", a);
    }

    if (x > val_pro) {
        printf("el numero %d esta por encima del promedio \n", x);
    } else {
        printf("el numero %d esta por debajo del promedio \n", x);
    }
    }
    return 0;
}
```

3.10.7. Salida del programa

```
C:\Users\MSI\Desktop\TDLP\teoria_de_la_programacion\PracticasC\trabajo_autonomo>ejercicio_50.exe
ingrese un numero entero:
20
ingrese un numero entero:
45
ingrese un numero entero:
31
el valor del promedio es: 32.00
el numero 20 esta por debajo del promedio
el numero 45 esta por encima del promedio
el numero 31 esta por debajo del promedio
```

3.11.1. Enunciado del algoritmo

EJ-146

Diseñe un algoritmo que declare una variable c (entero) y lea su valor por teclado. Use una constante simbólica LIMITE = 20. Evalúe la expresión lógica (c > 0 AND c < LIMITE). Asigne el resultado a una variable booleana es_valido. Evalúe también la expresión relacional (c == LIMITE / 2). Muestre por pantalla ambos resultados identificando cuál es dato y cuál es información.

3.11.2. Análisis del problema

Datos de entrada:

- LIMITE = 20
- Variable c

Proceso:

1. Declarar variable c como entero, la variable es_valido como booleano.
2. Leer la variable c por el teclado
3. Evaluar la expresión lógica (c > 0 AND c < LIMITE) asignando el resultado como es_valido y si es verdadera determinar que está en el límite, de no serlo determinar lo contrario.
4. Evaluar la expresión relacional (c == LIMITE / 2), determinar que la expresión es igual a la mitad del límite, de no serlo determinar lo contrario.

Salida:

c es el dato.
es_valido es una información y si está en el límite o no está en el límite.
el resultado de la expresión relacional es una información y si es igual a la mitad del límite o no lo es.

3.11.3. Diagrama de flujo



3.11.4. Pseudocódigo

```
INICIO

//declaración de variables y constantes

Definir c como entero;

Definir es_valido como logico;

Definir LIMITE = 20;

//entrada

Leer c;

//proceso
```

```
es_valido = (c > 0 AND c < LIMITE);
//evaluación y salida identificando el dato y la información
//dato
Escribir (“el dato ingresado es” + c)
//información
Si es_valido entonces
Escribir (c+ “ está dentro del límite”);
Sino
Escribir (c + “ esta fuera del límite”);
Si (c == LIMITE / 2) entonces
Escribir (c + “ es igual a la mitad del límite”)
Sino
Escribir (c+ “ no es igual a la mitad del límite”)
FIN
```

3.11.5. Prueba de escritorio

Caso	LIMITE	c	es_valido = (c > 0 AND c < LIMITE)	c == LIMITE / 2
Caso 1	20	50	Falso	Falso
Caso 2	20	10	Verdadero	Verdadero

3.11.6. Codificación en lenguaje C

trabajo_autonomo > C ejercicio_146.c > main()

```c
#include <stdio.h>
#include <stdbool.h>

int main() {
    //declaracion de datos
    int LIMITE = 20;
    int c = 0;
    bool es_valido = true;
    //entrada
    printf("ingrese un numero entero: \n");
    scanf("%d", &c);
    //proceso
    es_valido = (c > 0 && c < LIMITE);

    printf("el dato ingresado es: %d \n", c);
    //evaluacion de la condicion
    if (es_valido) {
        printf("el numero está dentro del limite \n");
    } else {
        printf("el numero está fuera del limite \n");
    }
    if (c == LIMITE / 2) {
        printf(" %d es igual a la mitad del limite", c);
    } else {
        printf(" %d no es igual a la mitad del limite", c);
    }
    return 0;
}
```

3.11.7. Salida del programa

```
C:\Users\MSI\Desktop\TDLP\teoria_de_la_programacion\PracticasC\trabajo_autonomo>gcc ejercicio_146.c -o ejercicio_146

C:\Users\MSI\Desktop\TDLP\teoria_de_la_programacion\PracticasC\trabajo_autonomo>ejercicio_146.exe
ingrese un numero entero:
50
el dato ingresado es: 50
el numero est├í fuera del limite
 50 no es igual a la mitad del limite
C:\Users\MSI\Desktop\TDLP\teoria_de_la_programacion\PracticasC\trabajo_autonomo>
```

3.12.1. Enunciado del algoritmo

EJ-147

Cree un algoritmo que lea dos números enteros c y n por teclado. Use una constante literal 2 para evaluar paridad. Calcule el cociente entero y el residuo usando operadores aritméticos. Evalúe las expresiones lógicas (c % 2 == 0) y (n % 2 == 0) para determinar paridad. Muestre por pantalla: cociente, residuo y si cada número es par o impar.

3.12.2. Análisis del problema

Datos de entrada:

- Variable c
- Variable n

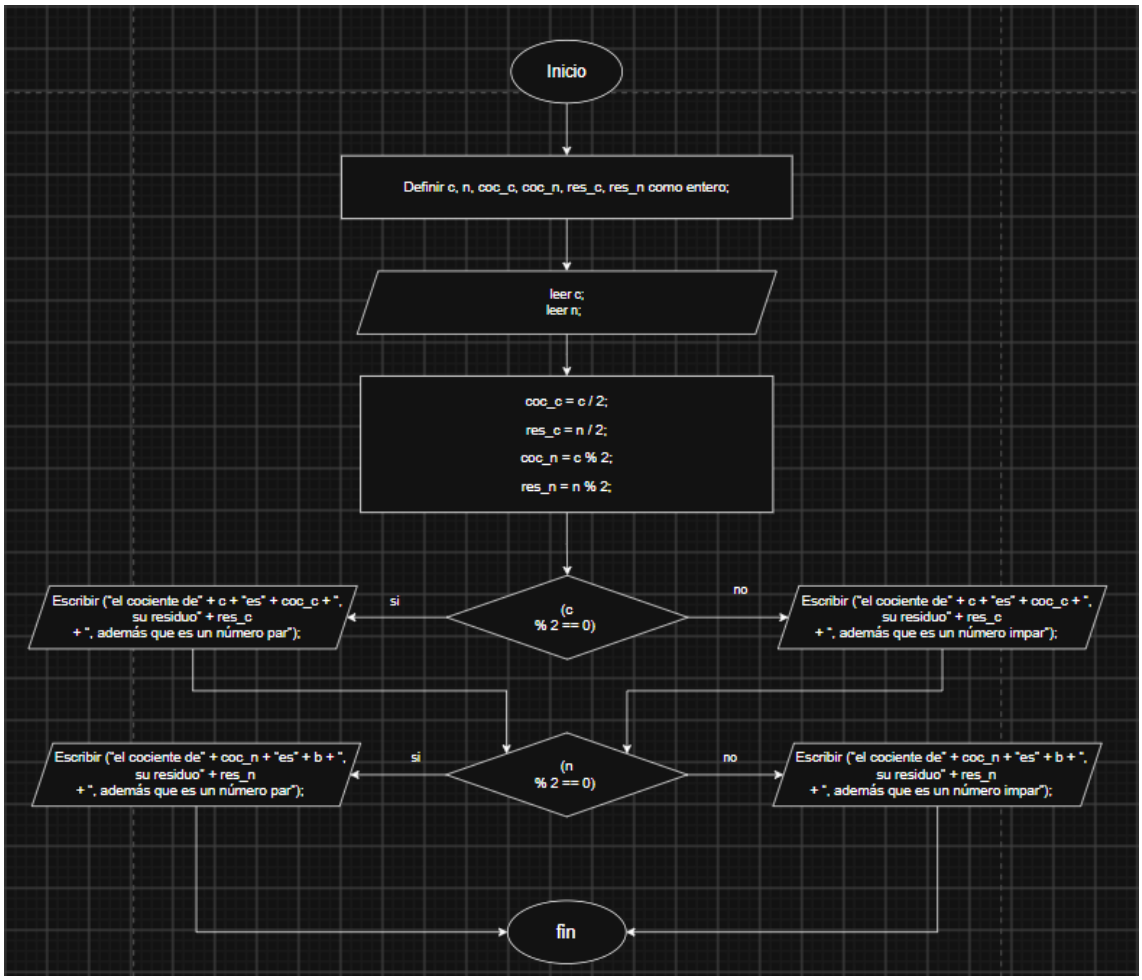
Proceso:

1. Declarar las variables n y c como enteros
2. Leer las variables n y c por teclado
3. Calcular el cociente entero y el residuo de cada variable usando la constante literal 2.
4. Evaluar la expresión lógica (c % 2 == 0), si esta es verdadera, determinar que la variable c es par, de ser contrario determinar que es impar.
5. Evaluar la expresión lógica (n % 2 == 0), si esta es verdadera, determinar que la variable c es par, de ser contrario determinar que es impar.

Salida

cociente y residuo de c, el número es par o impar.
cociente y residuo de n, el número es par o impar.

3.12.3. Diagrama de flujo



3.12.4. Pseudocódigo

```
INICIO
//declaración de variables
Definir c, n, coc_c, coc_n, res_c, res_n como entero;
//entrada
Leer n;
Leer c;
//proceso
coc_c = c / 2;
res_c = n / 2;
coc_n = c % 2;
res_n = n % 2;
// evaluación y salida de c
Si (c % 2 == 0) entonces;
      Escribir (“el cociente de” + c + “es” + coc_c + “, su residuo” + res_c
      + “, además que es un número par”);
```

```
Sino
      Escribir (“el cociente de” + c + “es” + coc_c + “, su residuo” + res_c
      + “, además que es un número impar”);
// evaluación y salida de n
Si (n % 2 == 0) entonces;
      Escribir (“el cociente de” + coc_n + “es” + b + “, su residuo” + res_n
      + “, además que es un número par”);
Sino
      Escribir (“el cociente de” + coc_n + “es” + b + “, su residuo” + res_n
      + “, además que es un número impar”);
FIN
```

3.12.5. Prueba de escritorio

Pasos	1	2	3	4	5	6	7	8	9
Casos	c	n	coc_c = c / 2	res_c = n / 2	coc_n = c % 2	res_n = n % 2	(c % 2 == 0)	(n % 2 == 0)	Salida
Caso 1	21	40	10	1	20	0	falso	verdadero	El cociente de 21 es 10, su residuo 1, además es impar El cociente de 40 es 20, su residuo 0, además es par
Caso 2	30	13	15	0	5	3	verdadero	falso	El cociente de 30 es 15, su residuo 0, además es par El cociente de 13 es 5, su residuo 3, además es impar

3.12.6. Codificación en lenguaje C

```c
#include <stdio.h>

int main() {
    //declaracion de datos
    int c, n, coc_c, coc_n, res_c, res_n = 0;
    //entrada
    printf("ingrese un numero entero para c: \n");
    scanf("%d", &c);
    printf("ingrese un numero entero para n: \n");
    scanf("%d", &n);
    //proceso
    coc_c = c / 2;
    res_c = c % 2;
    coc_n = n / 2;
    res_n = n % 2;
    //evaluacion
    if (res_c == 0) {
        printf("el cociente de %d es %d, su residuo %d, ademas es un numero par \n", c, coc_c, res_c);
    } else {
        printf("el cociente de %d es %d, su residuo %d, ademas es un numero impar \n", c, coc_c, res_c);
    }
    if (res_n == 0) {
        printf("el cociente de %d es %d, su residuo %d, ademas es un numero par \n", n, coc_n, res_n);
    } else {
        printf("el cociente de %d es %d, su residuo %d, ademas es un numero impar \n", n, coc_n, res_n);
    }
    return 0;
}

```

3.12.7. Salida del programa

```
C:\Users\MSI\Desktop\TDLP\teoria_de_la_programacion\PracticasC\trabajo_autonomo>ejercicio_147.exe
ingrese un numero entero para c:
21
ingrese un numero entero para n:
40
el cociente de 21 es 10, su residuo 1, ademas es un numero impar
el cociente de 40 es 20, su residuo 0, ademas es un numero par
```

3.13.1. Enunciado del algoritmo

EJ-148

Construya un algoritmo que lea el sueldo base (real) de un trabajador. Use una constante simbólica BONO = 0.5 (porcentaje de bono) y una constante literal 100 para el cálculo. Calcule el bono con la expresión aritmética (sueldo * BONO / 100) y el sueldo total con (sueldo + bono). Evalúe la expresión relacional (sueldo_total > sueldo). Muestre por pantalla el desglose completo.

3.13.2. Análisis del problema

Datos de entrada

- BONO = 0.5
- Sueldo

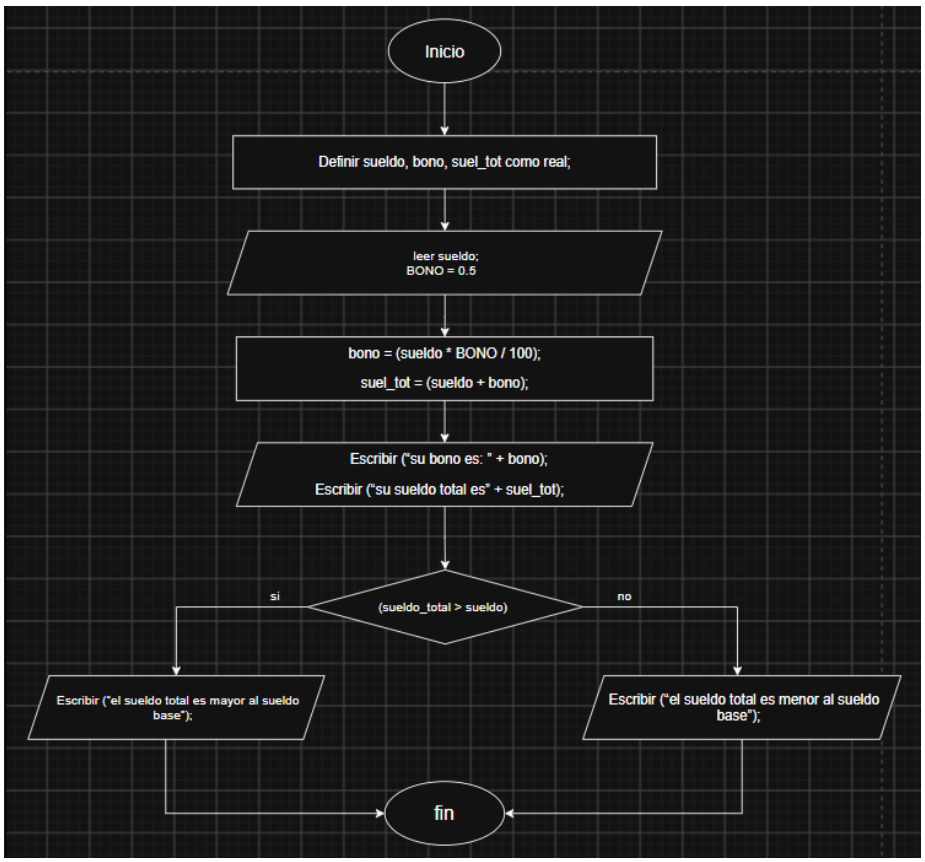
Proceso

1. Declarar sueldo, bono, sueldo total como real.
2. Leer sueldo por el teclado
3. Calcular el bono = (sueldo * BONO / 100).
4. Calcular sueldo_total = (sueldo + bono)
5. Evaluar (sueldo_total > sueldo).

Salida

bono
sueldo total
sueldo total mayor a sueldo base.

3.13.3. Diagrama de flujo



3.13.4. Pseudocódigo

```
INICIO
//declaración de variables
BONO = 0.5
Definir sueldo, bono, suel_tot como real;
//entrada
Escribir (“ingrese su sueldo base”);
Leer sueldo;
//proceso
       bono = (sueldo * BONO / 100);
       suel_tot = (sueldo + bono);
//salida
       Escribir (“su bono es: ” + bono);
       Escribir (“su sueldo total es” + suel_tot);
// evaluación de la expresión
Si (suel_tot > sueldo) entonces
       Escribir (“el sueldo total es mayor al sueldo base”);
Sino
```

```
        Escribir (“el sueldo total es menor al sueldo base”);
FIN
```

3.13.5. Prueba de escritorio

Pasos	1	2	3	4	5	6
Casos	BONO	sueldo	bono	Sueldo total	(suel_tot > sueldo	salida
Caso 1	0.5	460.50	2.30	462.80	Verdadero	Su bono es: 2.30 Su sueldo total es: 462.80 El sueldo total es mayor al sueldo base
Caso 2	0.5	523.00	2.62	525.62	Verdadero	Su bono es: 2.62 Su sueldo total es:525.62 El sueldo total es mayor al sueldo base

3.13.6. Codificación en lenguaje C

trabajo_autonomo > C ejercicio_148.c > main()

```c
#include <stdio.h>

void main() {
    //datos de entrada
    float BONO = 0.50;
    float sueldo, bono, suel_tot = 0.00;
    //entrada
    printf("ingrese su sueldo base\n");
    scanf("%f", &sueldo);
    //proceso
    bono = ((sueldo * BONO) / 100);
    suel_tot = (sueldo + bono);
    //salida y evaluacion de condiciones
    printf("su bono es: %.2f \n", bono);
    printf("su sueldo total es: %.2f \n", suel_tot);

    if (suel_tot > sueldo) {
        printf("el sueldo total es mayor al sueldo base");
    } else {
        printf("el sueldo total es menor al sueldo base");
    }
}
```

3.13.7. Salida del programa

```
C:\Users\MSI\Desktop\TDLP\teoria_de_la_programacion\PracticasC\trabajo_autonomo>ejercicio_148.exe
ingrese su sueldo base
460.50
su bono es: 2.30
su sueldo total es: 462.80
el sueldo total es mayor al sueldo base
C:\Users\MSI\Desktop\TDLP\teoria_de_la_programacion\PracticasC\trabajo_autonomo>
```

3.14.1. Enunciado del algoritmo

EJ-149

Diseñe un algoritmo que declare las variables m (entero) y y (carácter). Asigne los literales 20 y 'M' respectivamente. Convierta m a real y asígnelo a una nueva variable y. Calcule la expresión aritmética (y * 1.5) y asígnela a resultado. Evalúe la expresión lógica (y >= 'a' AND y <= 'z'). Muestre por pantalla todos los valores con sus tipos.

3.14.2. Análisis del problema

Datos de entrada

- m = 20
- y = 'M'

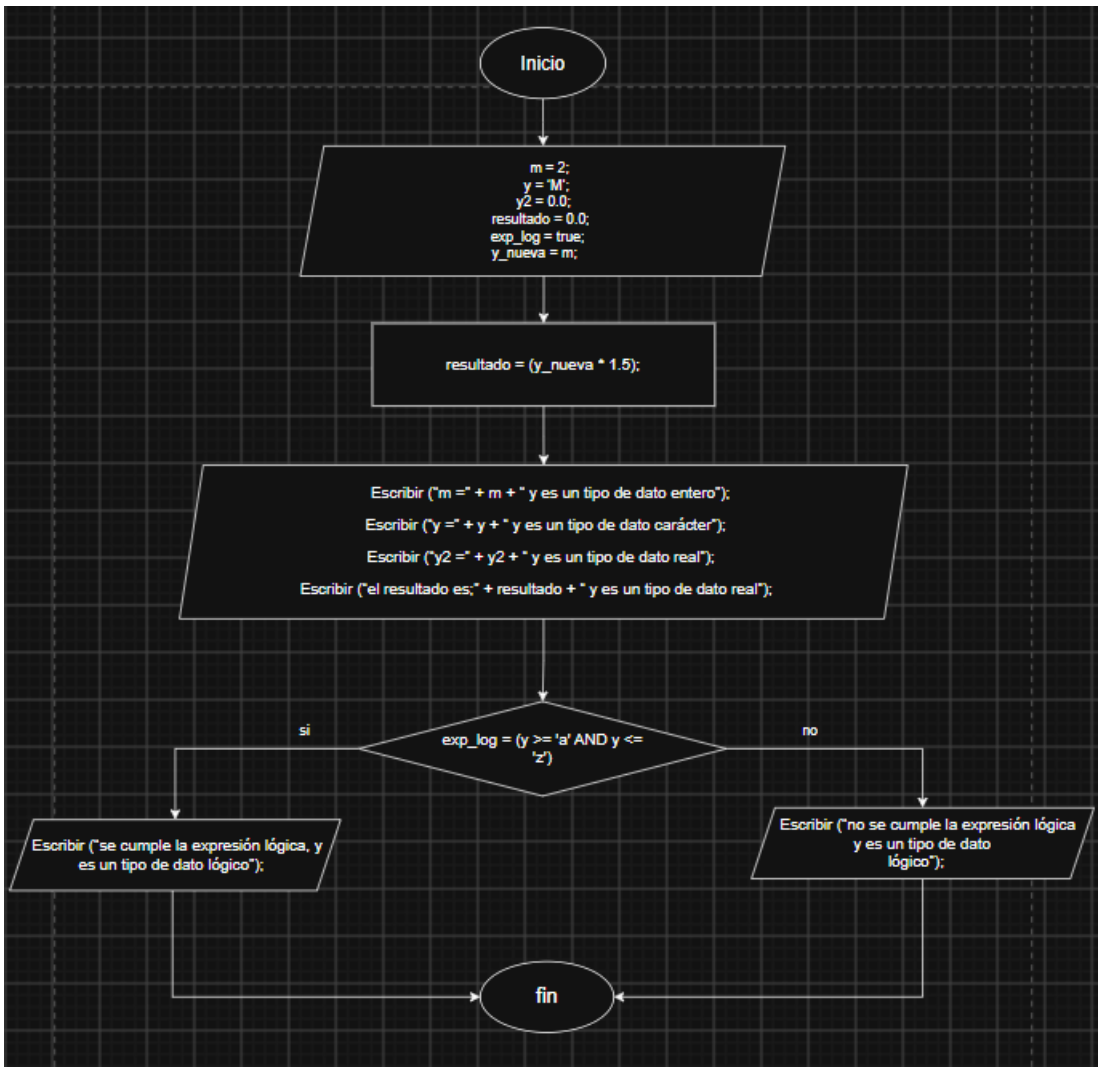
Proceso

1. declarar resultado y y2 como real
2. convertir m, a real y asignarlo a y2.
3. resultado = (y2 * 1.5).
4. evaluar (y >= 'a' AND y <= 'z')

Salida

Resultado de m con su tipo de dato
Resultado de y con su tipo de dato
Resultado de y2 con su tipo de dato
Resultado de resultado con su tipo de dato
Resultado de la expresión con su tipo de dato

3.14.3. Diagrama de flujo



3.14.4. Pseudocódigo

```
INICIO
//entrada
m = 20;
y = ‘M’;
y2 = 0.0;
resultado = 0.0;
exp_log = true;
//proceso
       y2 = m;
       resultado = (y2 * 1.5);
       exp_log = (y >= 'a' AND y <= 'z')
```

```
//salida y evaluación
      Escribir (“m =” + m + “ y es un tipo de dato entero”);
      Escribir (“y =” + y + “ y es un tipo de dato carácter”);
      Escribir (“y2 =” + y2 + “ y es un tipo de dato real”);
      Escribir (“el resultado es;” + resultado + “ y es un tipo de dato
      real”);
Si exp_log = (y >= 'a' AND y <= 'z') entonces
      Escribir (“se cumple la expresión lógica, y es un tipo de dato
      lógico”);
Sino
      Escribir (“no se cumple la expresión lógica y es un tipo de dato
      lógico”);
FIN
```

3.14.5. Prueba de escritorio

Pasos	1	2	3	4	5	6	7
Casos	m	y	y2	resultado	exp_log	¿Se cumple?	Salida
Caso 1	20	M	20	30	(y >= 'a' AND y <= 'z')	Falso	m = 20 y es un tipo de dato entero y = M, es un tipo de dato caracter y2 = 20.00 y es un tipo de dato real resultado = 30.00 y es un tipo de dato real se define como expresión lógica exp_log = (y >= 'a' AND y <= 'z'), entonces: no se cumple la expresión lógica y es un tipo de dato lógico
Caso 2	30	Y	30	45	(y >= 'a' AND y <= 'z')	Falso	m = 30 y es un tipo de dato entero y = Y, es un tipo de dato caracter y2 = 30.00 y es un tipo de dato real resultado = 45.00 y es un tipo de dato real se define como expresión lógica exp_log = (y >= 'a' AND y <= 'z'), entonces: no se cumple la expresión lógica y es un tipo de dato lógico
Nota: en este ejercicio se dio los datos de las variables, pero se ejecutó con otros datos para el caso 2.							

3.14.6. Codificación en lenguaje C

```c
trabajo_autonomo > C ejercicio_149.c > main()
#include <stdio.h>
#include <stdbool.h>

int main() {
    //datos de entrada
    char y = 'M';
    int m = 20;
    float y2, resultado = 0.0;
    bool exp_log = true;
    //proceso
    y2 = m;
    resultado = (y2 * 1.5);
    exp_log = (y >= 'a' && y <= 'z');
    //salida
    printf("m = %d y es un tipo de dato entero\n", m);
    printf("y = %c y es un tipo de dato carácter\n", y);
    printf("y2 = %.2f y es un tipo de dato real\n", y2);
    printf("resultado = %.2f y es un tipo de dato real\n", resultado);
    //evaluacion
    printf("se define como expresion logica\nexp_log = (y >= 'a' AND y <= 'z'), entonces:\n");
    if (exp_log) {
        printf("se cumple la expresion logica y es un tipo de dato logico");
    } else {
        printf("no se cumple la expresion logica y es un tipo de dato logico");
    }
    return 0;
}
```

3.14.7. Salida del programa

```
C:\Users\MSI\Desktop\TDLP\teoria_de_la_programacion\PracticasC\trabajo_autonomo>ejercicio_149.exe
m = 20 y es un tipo de dato entero
y = M y es un tipo de dato carácter
y2 = 20.00 y es un tipo de dato real
resultado = 30.00 y es un tipo de dato real
se define como expresion logica
exp_log = (y >= 'a' AND y <= 'z'), entonces:
no se cumple la expresion logica y es un tipo de dato logico
C:\Users\MSI\Desktop\TDLP\teoria_de_la_programacion\PracticasC\trabajo_autonomo>
```

3.15.1. Enunciado del algoritmo

EJ-150

Cree un algoritmo que lea la base (real) y la altura (real) de un triángulo. Use la constante literal 2 en el denominador. Calcule el área con la expresión aritmética ((base * altura) / 2) y asígnela a una variable area. Evalúe la expresión relacional (area > 0) y la expresión lógica (base > 0 AND altura > 0). Muestre por pantalla el área calculada y los resultados booleanos.

3.15.2. Análisis del problema

Datos de entrada

- Base

- Altura

Proceso

1. Declarar a base, altura, area como reales
2. Calcular area = ((base * altura) / 2)
3. Evaluar (area > 0)
4. Evaluar (base > 0 AND altura > 0)

Salida

area
area mayor a cero
base mayor a cero y altura mayor a cero

3.15.3. Diagrama de flujo



3.15.4. Pseudocódigo

```
INICIO
//declaración de variables
Definir base, altura, area como reales;
// variables
Leer base; //en metros
Leer altura; //en metros
//proceso
      area = (base * altura) / 2;
//salida
      Escribir (“el área del triángulo es:” + area);
//evaluación
Si (area > 0) entonces
      Escribir (“el área es mayor a cero”);
Sino
      Escribir (“el área es menor a cero”);
Si (base > 0 AND altura > 0) entonces
      Escribir (“la base y la altura son mayores a cero”)
sino
      Escribir (“la base y la altura son menores a cero”)
FIN
```

3.15.5. Prueba de escritorio

Pasos	1	2	3	4	5	6
Casos	Base	Altura	área	(area > 0)	(base > 0 AND altura > 0)	Salida
Caso 1	22	30	330	verdadero	verdadero	el área del triángulo es: 330.00 metros cuadrados el área es mayor a cero la base y la altura son mayores a cero
Caso 2	12.6	21.3	134.19	verdadero	verdadero	el área del triángulo es: 134.19 metros cuadrados el área es mayor a cero la base y la altura son mayores a cero

3.15.6. Codificación en lenguaje C

Bienvenido 50.c > main()

```c
#include <stdio.h>

int main() {
    //declaracion de datos
    float base, altura, area = 0.0;
    //entrada de datos
    printf("ingrese la base del triangulo en metros: \n");
    scanf("%f", &base);
    printf("ingrese la altura del triangulo en metros: \n");
    scanf("%f", &altura);
    //proceso
    area = (base * altura)/2;
    //salida y evaluacion
    printf("el area del triangulo es: %.2f metros cuadrados\n", area);

    if (area > 0) {
        printf("el area es mayor a cero \n");
    } else {
        printf("el area es menor a cero \n");
    }
    if (base > 0 && altura > 0) {
        printf("la base y la altura son mayores a cero \n");
    } else {
        printf("la base y la altura son menores a cero \n");
    }
    return 0;
}
```

3.15.7. Salida del programa

```
C:\Users\MSI\Desktop\TDLP\teoria_de_la_programacion\unidad_1\AA\trabajo_autonomo>gcc ejercicio_150.c -o ejercicio_150.exe

C:\Users\MSI\Desktop\TDLP\teoria_de_la_programacion\unidad_1\AA\trabajo_autonomo>ejercicio_150.exe
ingrese la base del triangulo en metros:
22
ingrese la altura del triangulo en metros:
30
el area del triangulo es: 330.00 metros cuadrados
el area es mayor a cero
la base y la altura son mayores a cero
```

3.16.1. Enunciado del algoritmo

EJ-246

Declare las variables p (entero) y dato2 (real). Asigne a p el valor literal 15 y a dato2 el valor literal 1.5. Realice las siguientes operaciones secuenciales: sume ambas variables asignando el resultado a una nueva variable suma (¿qué tipo debe tener?), luego multiplique suma por 2 y asígnela a la variable duplicado. Muestre por pantalla los valores de las variables en cada paso. Justifique el tipo de dato de cada variable.

3.16.2. Análisis del problema

Datos de entrada:

- P = 15
- dato2 = 1.5

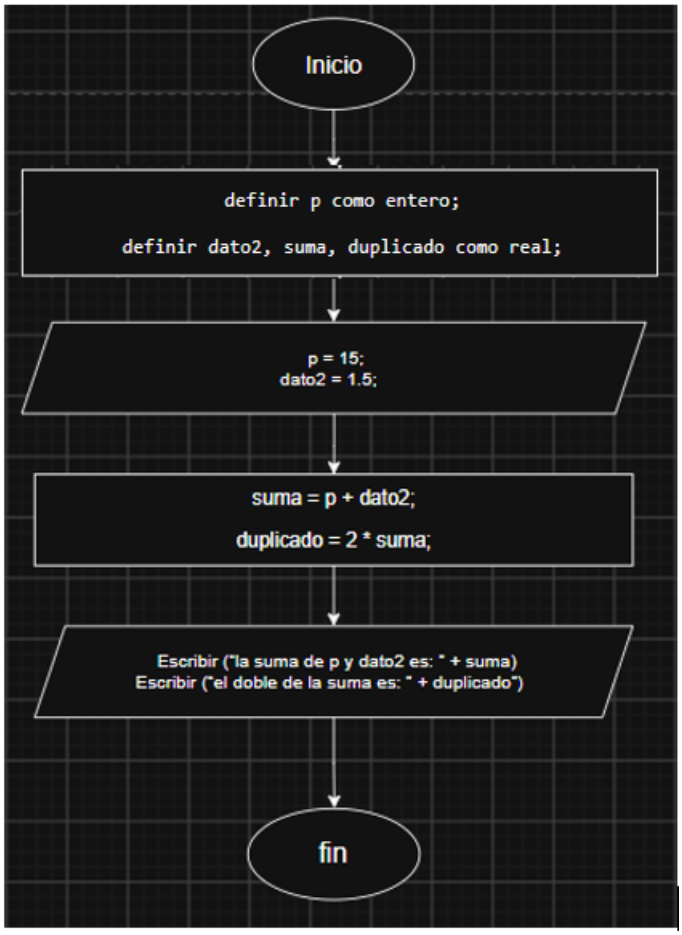
Proceso

1. declara p como entero, dato2, suma y duplicado como real
2. asignar suma = p + dato2
3. asignar duplicado = 2 * suma

Salida

suma
duplicado

3.16.3. Diagrama de flujo



3.16.4. Pseudocódigo

```
INICIO
//declaración de variables
Definir p como entero; //p es número entero porque así lo pide el problema.
Definir dato2, suma, duplicado como real; // dato2 es real porque lo pide el
problema, suma porque al sumar las variables da un número decimal y duplicado
porque se va a multiplicar por un numero real.
//entrada
      p = 15;
      dato2 = 1.5;
//proceso
      suma = p + dato2;
      duplicado = 2 * suma;
//salida
      Escribir (“la suma de p y dato2 es: ” + suma)
      Escribir (“el doble de la suma es: ” + duplicado”)
FIN.
```

3.16.5. Prueba de escritorio

Pasos	1	2	3	4	5
Casos	p	Dato2	suma	duplicado	salida
Caso 1	15	1.5	16.50	33.00	la suma de p y dato2 es: 16.50 el doble de la suma es: 33.00
Caso 2	25	2.5	27.50	55.00	la suma de p y dato2 es: 27.50 el doble de la suma es: 55.00
Nota: el ejercicio ya nos da los datos para las variables así que se modificaron los datos dentro del código para el caso 2.					

3.16.6. Codificación en lenguaje C

ejercicio_246.c > main()

```c
#include <stdio.h>

void main() {
    //declaracion de variables
    int p = 0;
    float dato2, suma, duplicado = 0.0;
    //entrada
    p = 15;
    dato2 = 1.5;
    //proceso
    suma = p + dato2;
    duplicado = suma * 2;;
    //salida
    printf("p = %d \ndato2 = %.1f \n", p, dato2);
    printf("la suma de p y dato2 es: %.2f \n", suma);
    printf("el doble de la suma es: %.2f \n", duplicado);
}
```

3.16.7. Salida del programa

```
C:\Users\MSI\Desktop\TDLP\teoria_de_la_programacion\unidad_1\AA\trabajo_autonomo>ejercicio_246.exe
p = 15
dato2 = 1.5
la suma de p y dato2 es: 16.50
el doble de la suma es: 33.00
```

3.17.1. Enunciado del algoritmo

EJ-247

Construya un algoritmo que lea una temperatura en grados Celsius (real) desde teclado. Use una constante simbólica DESCUENTO con valor 1.5 para representar el factor 9/5. Calcule la temperatura en Fahrenheit usando la expresión aritmética: (Celsius * DESCUENTO) + 32. Asigne el resultado a la variable fahrenheit. Muestre por pantalla un mensaje del

tipo: "X °C equivalen a Y °F". Identifique cuál es el dato de entrada y cuál es la información de salida.

3.17.2. Análisis del problema

Datos de entrada

- Temperatura en Celsius

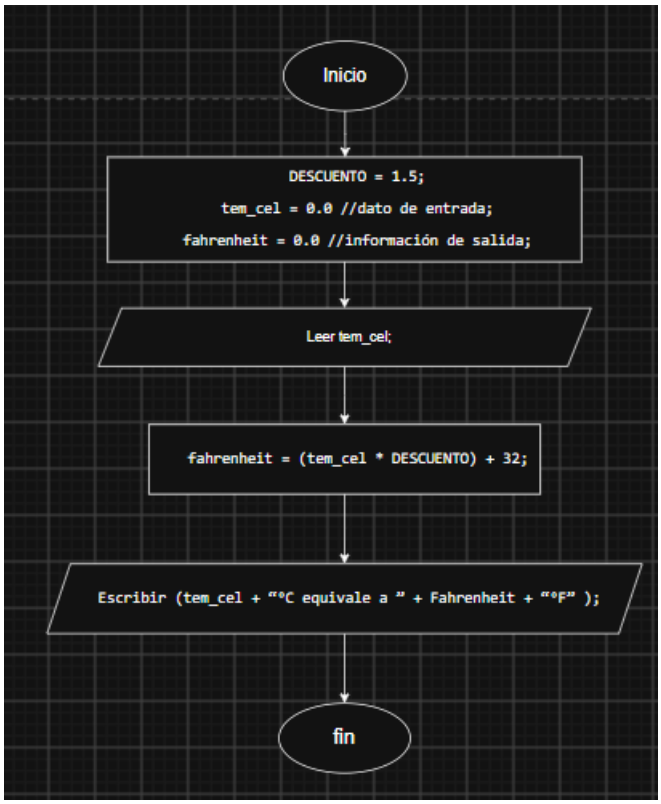
Proceso

1. Declarar constante DESCUENTO = 1.5 y la variable temperatura en Celsius como real.
2. Leer temperatura en Celsius desde el teclado.
3. Calcular Fahrenheit = (Celsius * DESCUENTO) + 32 y asignarle la variable fahrenheit.

Salida

Temperatura en Celsius equivale a temperatura Fahrenheit.

3.17.3. Diagrama de flujo



3.17.4. Pseudocódigo

```
INICIO
//declaración de constantes y variables
DESCUENTO = 1.5;
tem_cel = 0.0; //dato de entrada
fahrenheit = 0.0; //información de salida
//entrada
   Leer tem_cel;
//proceso
```

```
   fahrenheit = (tem_cel * DESCUENTO) + 32;
//salida
   Escribir (tem_cel + “°C equivale a ” + Fahrenheit + “°F” );
FIN.
```

3.17.5. Prueba de escritorio

Pasos	1	2	3	4
Casos	DESCUENTO	Celsius	fahrenheit	Salida
Caso 1	1.5	37.00	87.50	37 grados Celsius equivalen a 87.5 grados fahrenheit
Caso 2	1.5	25.00	69.50	25.00 grados Celsius, equivalen a 69.50 grados farenheit

3.17.6. Codificación en lenguaje C

trabajo_autonomo > C ejercicio_247.c > main()

```c
#include <stdio.h>

void main() {
    //declaración de constantes y variables
    float DESCUENTO = 1.5;
    float tem_cel = 0.0; //dato de entrada;
    float fahrenheit = 0.0; //información de salida;
    //entrada
    printf("ingrese la temperatura en Celsius:\n");
    scanf("%f", &tem_cel);
    //proceso
    fahrenheit = (tem_cel * DESCUENTO) + 32;
    //salida
    printf("%.2f grados Celsius, equivalen a %.2f grados farenheit", tem_cel, fahrenheit);
}
```

3.17.7. Salida del programa

```
C:\Users\MSI\Desktop\TDLP\teoria_de_la_programacion\PracticasC\trabajo_autonomo>gcc ejercicio_247.c -o ejercicio_247

C:\Users\MSI\Desktop\TDLP\teoria_de_la_programacion\PracticasC\trabajo_autonomo>ejercicio_247.exe
ingrese la temperatura en Celsius:
37
37.00 grados Celsius, equivalen a 87.50 grados farenheit
C:\Users\MSI\Desktop\TDLP\teoria_de_la_programacion\PracticasC\trabajo_autonomo>
```

3.18.1. Enunciado del algoritmo

EJ-248

Diseñe un algoritmo que declare las variables y (entero), c (real) y p (carácter). Asigne los literales 30, 2.0 y 'Z' respectivamente. Evalúe las siguientes expresiones: aritmética (y + int(c)), relacional (y > 10), lógica (y > 10 AND p == 'Z'). Muestre cada resultado identificando el tipo de dato devuelto por cada expresión.

3.18.2. Análisis del problema

Datos de entrada

- y = 30
- c = 2.0
- p = ‘Z’

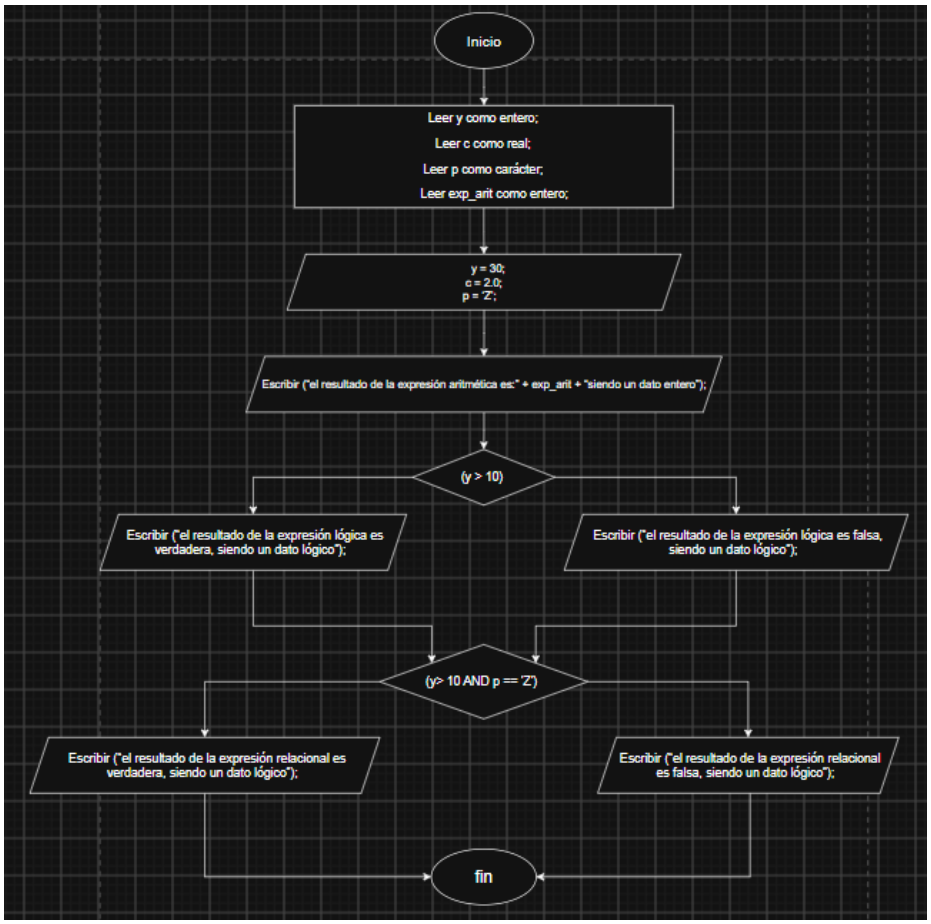
Proceso

1. Declarar las variables y como entero, c como real, p como carácter y q como entero.
2. Evaluar la expresión (y + int(c))
3. Evaluar la expresión (y > 10)
4. Evaluar la expresión (y > 10 AND p == 'Z')

Salida

Resultado de la expresion aritmética siendo un dato entero
Resultado de la expresion logica siendo un dato lógico
Resultado de la expresión relacional siendo un dato lógico

3.18.3. Diagrama de flujo



3.18.4. Pseudocódigo

```
INICIO
//declaración de variables
Leer y como entero;
Leer c como real;
Leer p como carácter;
Leer exp_arit como entero;
//entrada
y = 30;
c = 2.0;
p = ‘Z’;
//proceso
      exp_arit = (y + c);
//evaluación y salida
      Escribir (“el resultado de la expresión aritmética es:” + exp_arit +
      “siendo un dato entero”);
Si (y > 10) entonces
      Escribir (“el resultado de la expresión lógica (y > 10)  es verdadera,
      siendo un dato lógico”);
Sino
      Escribir (“el resultado de la expresión lógica es falsa, siendo un
      dato lógico”);
Si (y > 10 AND p == 'Z') entonces
      Escribir (“el resultado de la expresión relacional (y > 10 AND p ==
      'Z') es verdadera, siendo un dato lógico”);
Sino
      Escribir (“el resultado de la expresión relacional (y > 10 AND p ==
      'Z') es falsa, siendo un dato lógico”);

FIN.
```

3.18.5. Prueba de escritorio

Pasos	1	2	3	4	5	6	7
Casos	y	c	p	exp_arit	(y > 10)	(y > 10 AND p == 'Z')	Salida
Caso 1	30	2.0	Z	32.00	verdadero	verdadero	Resultado: 32.00 (dato entero) Primera expresión: verdadera (dato lógico) Segunda expresión: verdadera (dato lógico)
Caso 2	45	3.1	F	48.10	verdadero	falsa	Resultado: 48.10 (dato entero) Primera expresión: verdadera (dato lógico) Segunda expresión: falsa (dato lógico)
Nota: el ejercicio ya dio los datos de las variables para el caso 2 modificamos tales datos en el código.							

3.18.6. Codificación en lenguaje C

```c
#include <stdio.h>

void main() {
    //declaracion de datos
    int y = 0;
    float c = 0.0;
    char p ;
    int exp_arit = 0;
    //entrada
    y = 30;
    c = 2.0;
    p = 'Z';
    //proceso
    exp_arit = (y + c);
    //evaluacion y salida
    printf("y = 30 \nc = 2.0 \np = 'Z'\n");
    printf("el resultado de la expresión aritmética es: %d, siendo un dato entero\n", exp_arit);
    if (y > 10) {
        printf("el resultado de la expresión lógica (y > 10) es verdadera, siendo un dato lógico\n");
    } else {
        printf("el resultado de la expresión lógica (y > 10) es falsa, siendo un dato lógico\n");
    }
    if (y > 10 && p == 'Z') {
       printf("el resultado de la expresión relacional (y > 10 AND p == 'Z') es verdadera, siendo un dato lógico\n");
    } else {
        printf("el resultado de la expresión relacional (y > 10 AND p == 'Z') es falsa, siendo un dato lógico\n");
    }

}
```

3.18.7. Salida del programa

```
C:\Users\MSI\Desktop\TDLP\teoria_de_la_programacion\PracticasC\trabajo_autonomo>gcc ejercicio_248.c -o ejercicio_248

C:\Users\MSI\Desktop\TDLP\teoria_de_la_programacion\PracticasC\trabajo_autonomo>ejercicio_248.exe
y = 30
c = 2.0
p = 'Z'
el resultado de la expresi├│n aritm├®tica es: 32, siendo un dato entero
el resultado de la expresi├│n l├│gica (y > 10) es verdadera, siendo un dato l├│gico
el resultado de la expresi├│n relacional (y > 10 AND p == 'Z') es verdadera, siendo un dato l├│gico

C:\Users\MSI\Desktop\TDLP\teoria_de_la_programacion\PracticasC\trabajo_autonomo>
```

3.19.1. Enunciado del algoritmo

EJ-249

Cree un algoritmo que calcule el área de un círculo. Declare una constante simbólica PI con valor 3.14159 y una variable real radio. Lea el radio desde teclado. Calcule el área usando la expresión aritmética (PI * radio * radio) y asígnela a la variable area. Muestre por pantalla el resultado con un mensaje informativo. Justifique por qué se usa una constante en lugar de una variable para PI.

Justificación: se usa PI como constante porque es un valor fijo, además al tener infinitos decimales es imposible escribir todos.

3.19.2. Análisis del problema

Datos de entrada

- radio

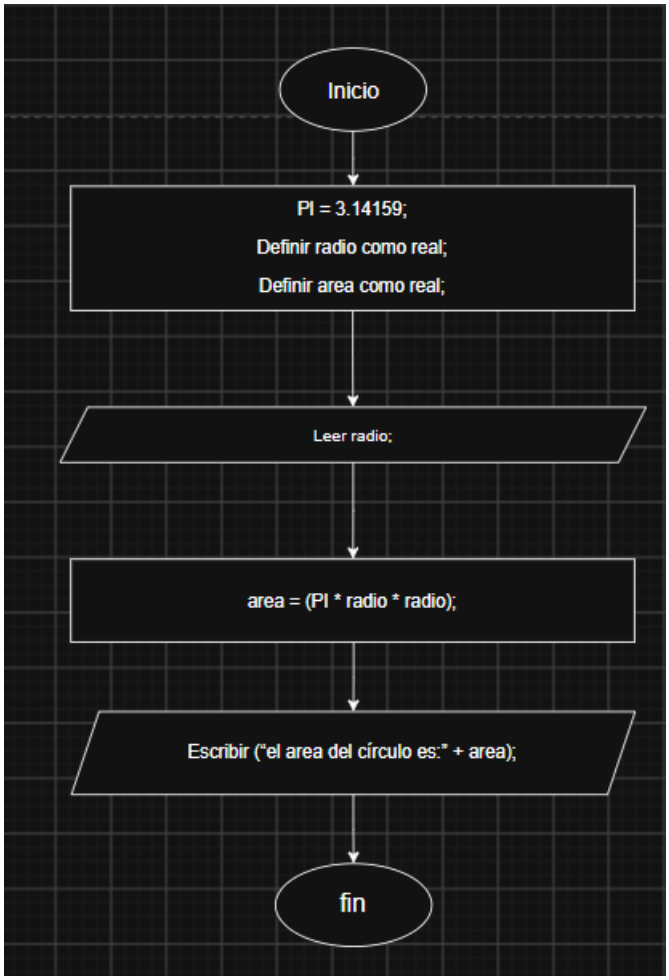
Proceso

1. declarar constante PI = 3.14159, variable radio y area como real.
2. Calcular area = (PI * radio * radio)

Salida

Resultado del area

3.19.3. Diagrama de flujo



3.19.4. Pseudocódigo

```
INICIO
//declaración de constantes y variables
PI = 3.14159;
Definir radio como real;
Definir area como real;
//entrada
      Escribir ("ingrese el radio del círculo");
Leer radio;
//proceso
      area = (PI * radio * radio);
//salida
```

```
      Escribir (“el area del círculo es:” + area);
FIN
```

3.19.5. Prueba de escritorio

Pasos	1	2	3	4
Casos	PI	radio	Área	Salida
Caso 1	3.14159	5	78.54	el área de la circunferencia es: 78.54 metros cuadrados
Caso 2	3.14159	4.6	66.48	el área de la circunferencia es: 66.48 metros cuadrados

3.19.6. Codificación en lenguaje C

ejercicio_249.c > main()

```c
#include <stdio.h>

void main() {
    //declaracion de datos
   float PI = 3.14159;
   float radio = 0.0;
   float area = 0.0;
   //entrada
   printf("ingrese el radio de la circunferencia en metros:\n");
   scanf("%f", &radio);
   //proceso
   area = (PI * radio * radio);
   //salida
   printf("el area de la circunferencia es: %.2f metros cuadrados", area);
}
```

3.19.7. Salida del programa

```
C:\Users\MSI\Desktop\TDLP\teoria_de_la_programacion\unidad_1\AA\trabajo_autonomo>gcc ejercicio_249.c -o ejercicio_249

C:\Users\MSI\Desktop\TDLP\teoria_de_la_programacion\unidad_1\AA\trabajo_autonomo>ejercicio_249.exe
ingrese el radio de la circunferencia en metros:
5
el area de la circunferencia es: 78.54 metros cuadrados
```

3.20.1. Enunciado del algoritmo

EJ-250

Declare una variable entera val2 y asígnele el valor literal 10. Lea otra variable entera dato1 por teclado. Calcule la expresión aritmética (val2 * dato1 + val2 - dato1) y asígnela a una variable resultado. Luego evalúe la expresión relacional (resultado >= 0) y la expresión lógica (resultado > 100 OR val2 > dato1). Muestre todos los valores por pantalla.

3.20.2. Análisis del problema

Datos de entrada

- val2 = 10
- dato1

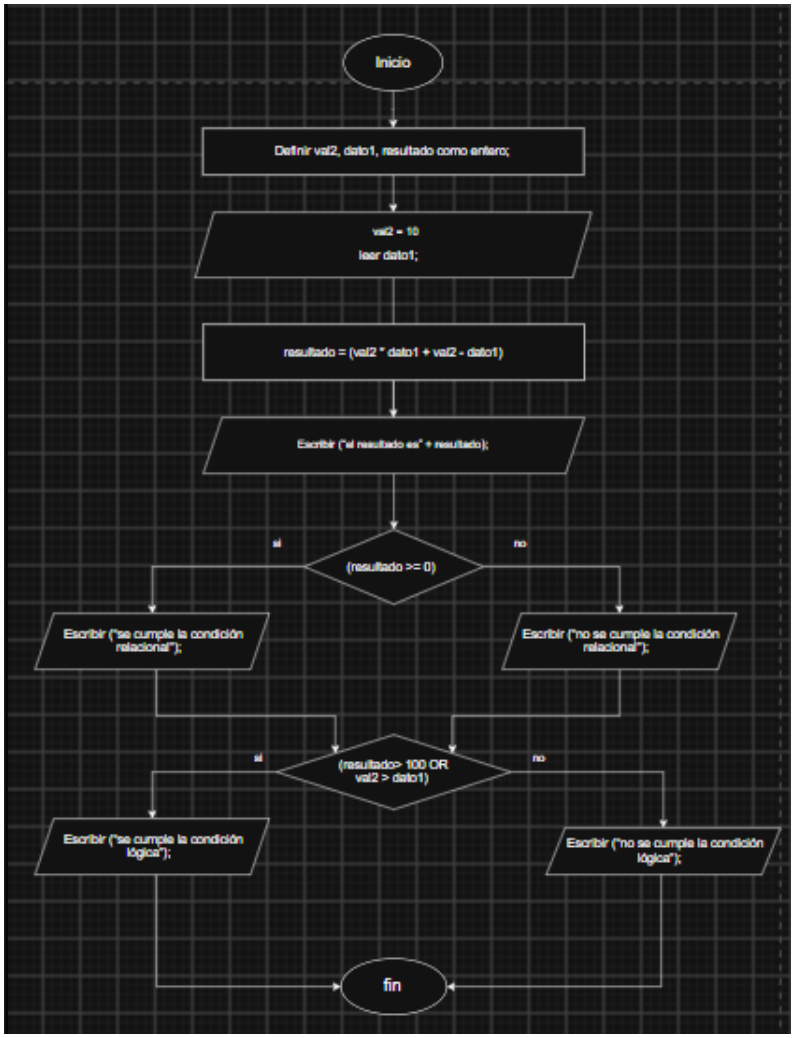
Proceso

1. declarar val2, dato1, resultado como entera.
2. leer dato1 por el teclado
3. resultado = (val2 * dato1 + val2 - dato1)
4. evaluar (resultado >= 0)
5. evaluar (resultado > 100 OR val2 > dato1)

Salida

Resultado
Evaluación de la expresión relacional
Evaluación de la expresión lógica

3.20.3. Diagrama de flujo



3.20.4. Pseudocódigo

```
INICIO
//declaración de las variables
Definir val2, dato1, resultado como entero;
//entrada
```

```
val2 = 10;
leer dato1;
//proceso
      resultado = (val2 * dato1 + val2 - dato1);
//evaluación y salida
      Escribir (“el resultado es” + resultado);
Si (resultado >= 0) entonces
      Escribir (“se cumple la condición relacional: (resultado >= 0)”);
Sino
      Escribir (“no se cumple la condición relacional (resultado >= 0)”);
Si (resultado > 100 OR val2 > dato1) entonces
      Escribir (“se cumple la condición lógica (resultado > 100 OR val2 >
      dato1)”);
Sino
      Escribir (“no se cumple la condición lógica (resultado > 100 OR val2 >
      dato1)”);
FIN.
```

3.20.5. Prueba de escritorio

Pasos	1	2	3	4	5	6
Casos	val2	dato1	resultado	(resultado >= 0)	(resultado > 100 OR val2 > dato1)	Salida
Caso 1	10	20	190	Verdadero	Verdadero	Resultado: 190 Se cumple la condición lógica Se cumple la condición relacional
Caso 2	10	5	45	Verdadero	Verdadero	Resultado: 45 Se cumple la condición lógica Se cumple la condición relacional

3.20.6. Codificación en lenguaje C

trabajo_autonomo > C ejercicio_250.c > main()

```c
#include <stdio.h>

void main() {
    //declaracion de datos
    int val2, dato1, resultado = 0;
    //entrada
    val2 = 10;
    printf("ingrese numero para dato1\n");
    scanf("%d", &dato1);
    //proceso
    resultado = (val2 * dato1 + val2 - dato1);
    //evaluacion y salida
    printf("el resultado es: %d\n", resultado);
    if (resultado >= 0) {
        printf("se cumple la condición relacional: (resultado >= 0)\n");
    } else {
        printf("no se cumple la condición relacional (resultado >= 0)\n");
    }
    if (resultado > 100 || val2 > dato1) {
        printf("se cumple la condición lógica (resultado > 100 || val2 > dato1)\n");
    } else {
        printf("no se cumple la condición lógica (resultado > 100 || val2 > dato1)\n");
    }
}
```

3.20.7. Salida del programa

```
C:\Users\MSI\Desktop\TDLP\teoria_de_la_programacion\PracticasC\trabajo_autonomo>gcc ejercicio_250.c -o ejercicio_250

C:\Users\MSI\Desktop\TDLP\teoria_de_la_programacion\PracticasC\trabajo_autonomo>ejercicio_250.exe
ingrese numero para dato1
20
el resultado es: 190
se cumple la condici├│n relacional: (resultado >= 0)
se cumple la condici├│n l├│gica (resultado > 100 || val2 > dato1)

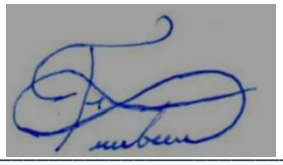
C:\Users\MSI\Desktop\TDLP\teoria_de_la_programacion\PracticasC\trabajo_autonomo>
```

5. Conclusiones de los algoritmos

1. El desarrollo de un programa necesita la elaboración de un plano para poder programarlo como lo son el pseudocódigo o el diagrama de flujo y también se necesita comprobar su funcionamiento mediante pruebas de escritorio, de no ser el caso podría haber errores o simplemente el código no funcionaría.
2. La elaboración de programas con nivel avanzado permite desarrollar un mejor razonamiento para su solución, lo que proporciona una mejora en el pensamiento racional y crítico, además de facilitar la resolución de diversos problemas de cualquier tipo.

Declaración de autoría

Declaro que el presente trabajo es de mi **autoría**, que **no se utilizó inteligencia artificial** (ni otra herramienta generadora de contenido) ni copia de terceros para su elaboración, y que conozco y acepto las sanciones establecidas en el reglamento institucional en caso de incumplimiento.



Firma del estudiante **Docente**

Enlace git:
https://github.com/fabiangordillo03/teoria_de_la_programacion